



by referring to these pdf files you need to answer my question bank questions according to marks allotted. once you read and understand these files, I'll send question bank file having 29 two-mark questions and 38 10-mark questions. you need to generate solution set for whole question bank in single response in text interface.

all answers should be able to score full allotted marks. make solution set simpler and easy to study and understand. don't use complicated terms if at all unavoidable the explain its meaning. treat me as a beginner not advanced database expert.

I've read and understood both Module 3 (Relational Database Design) and Module 4 (Advanced Database Systems) from your PDFs. I'm ready to receive your question bank — go ahead and share the 29 two-mark questions and 38 ten-mark questions, and I'll generate a complete solution set in one response, written in simple, beginner-friendly language.

what about module 1 and 2 pdf's?

You're right! I can see Module 3 and Module 4 PDFs are visible in our conversation, but Module 1 and Module 2 PDFs were also uploaded. Let me check if I can access their content.

I can see the file paths:

- `/mnt/user-data/uploads/Module__1_DBMS.pdf`
- `/mnt/user-data/uploads/Module_2_DBMS.pdf`

But their content didn't appear in the conversation context like Module 3 and 4 did. Let me read them now.

Ran 7 commands, viewed a file ✓



Reading skill for PDF files



Extract text from Module 1 and Module 2 PDFs

Script



Read more of Module 1

Script



Read remaining Module 1 content

Script



Read remaining relational algebra content from Module 1

Script



Read Module 2 PDF content

Script



Read more of Module 2

Script



Read remaining Module 2 content - triggers, procedures, query optimization

Script



Done

I've now read all four modules. Here's a complete summary of what I've covered:

Module 1 — Introduction to Database Modelling and Relational Algebra: Database basics, DBMS concepts, 3-schema architecture, data independence, file system vs DBMS, ER model (entities, attributes, relationships, keys, cardinality), Relational Algebra (Select, Project, Union, Intersection, Difference, Cartesian Product, Join, Division).

Module 2 — Fundamentals of SQL and Query Optimization: DDL/DML/DCL/TCL commands, SQL constraints (NOT NULL, PRIMARY KEY, UNIQUE, FOREIGN KEY, CHECK, DEFAULT), SQL operators (arithmetic, comparison, logical, set), Aggregate

functions (COUNT, SUM, AVG, MAX, MIN), Views, Triggers, Stored Procedures, Functions.

Module 3 — Relational Database Design: Informal design guidelines, Functional Dependencies, Armstrong's inference rules, closure, Normalization (1NF, 2NF, 3NF, BCNF), candidate keys, prime/non-prime attributes, update anomalies, spurious tuples.

Module 4 — Advanced Database Systems: OODBMS, Deductive DBMS, Spatial DBMS, Temporal DBMS, Statistical databases, Data Warehousing, OLAP operations, Multimedia databases, NoSQL databases.

I'm fully ready now. Please share your question bank with all 29 two-mark questions and 38 ten-mark questions, and I'll generate the complete solution set in one response!

PART-A (Each carries 2 marks)

1. Define schema and instance with an example.
2. Differentiate between selection (σ) and projection (π) in relational algebra?
3. Given a table STUDENT(SID, NAME, AGE, DEPT)
4. Create a view to display only student names and depts.
5. What is the purpose of query optimization?
6. Convert relational algebra $\sigma(A > 50)$ into SQL query.
7. Define checkpointing.
8. Consider the following relation:

EMP_ID	NAME	SALARY	DEPT	PROJECT
1	A	50000	CS	P1
2	B	60000	EE	P2
3	A	50000	CS	P3
4	C	70000	IT	P2
5	D	80000	EC	P4

Validate the following FDs:
EMP_ID $\rightarrow$ NAME
DEPT $\rightarrow$ PROJECT
List any two types of dependencies.

9. Define Multimedia Database with an example.
10. Define Temporal and Spatial DBMS.
11. What is data independence? Differentiate between logical and physical?

14. Define Query optimization.
15. Check if relation is in 2NF:
 $R(A,B,C), A \rightarrow B, AB \rightarrow C$

16. Consider the following relation:

BOOK_ID	TITLE	AUTHOR	PRICE	PUBLISHER
1	DBMS	X	500	P1
2	AI	Y	600	P2
3	DBMS	X	500	P1
4	ML	Z	700	P3
5	AI	Y	600	P2

Validate the following FDs:
BOOK_ID $\rightarrow$ TITLE
TITLE $\rightarrow$ BOOK_ID

17. Define recoverability in transactions.
18. Give any 2 examples of spatial queries.
19. Define valid time and transaction time?
20. Define three-schema architecture? Mention its levels.
21. Define Entity-Relationship (ER) model.
22. Find the result of $R \bowtie S$ if:

STU_ID	NAME	MARKS	DEPT	COURSE
1	A	78	CS	C1
2	B	60	EE	C1
3	A	78	CS	C2
4	B	60	EE	C3
5	C	80	IT	C3
6	D	80	EC	C2

Validate the following FDs:
STU_ID $\rightarrow$ NAME
DEPT $\rightarrow$ COURSE

first answer for part A (2 marks (29))

PART-A SOLUTIONS (2 Marks Each)

Q1. Define Schema and Instance with an example.

Schema is the structure or design of a database — it defines what tables exist, what columns they have, and what rules apply. It rarely changes.

Instance is the actual data stored in the database at a particular point in time. It changes frequently.

Example:

- Schema: `STUDENT(SID, NAME, AGE, DEPT)` — this is the structure
- Instance: The actual rows like (1, "Ram", 20, "CS") stored right now

Think of Schema as a **blueprint of a house** and Instance as **the furniture inside it at a given moment**.

Q2. Differentiate between Selection (σ) and Projection (π) in Relational Algebra.

Feature	Selection (σ)	Projection (π)
Works on	Rows (tuples)	Columns (attributes)
Purpose	Filters rows based on a condition	Picks specific columns
Duplicates	Keeps all matching rows	Removes duplicate rows
Example	$\sigma(\text{DEPT}='CS')(\text{STUDENT})$ — gives all CS students	$\pi(\text{NAME}, \text{AGE})(\text{STUDENT})$ — gives only name and age columns

Simple way to remember: σ = filter rows, π = pick columns.

Q3. Given STUDENT(SID, NAME, AGE, DNUM) — Create a view to display only student names and dnum.

```
sql
CREATE VIEW StudentView AS
SELECT NAME, DNUM
FROM STUDENT;
```

To query this view:

```
sql
SELECT * FROM StudentView;
```

A **view** is like a virtual table — it doesn't store data itself, it just shows selected columns from the original table.

Q4. What is the purpose of Query Optimization?

Query optimization is the process of finding the **most efficient way to execute a SQL query** so that it uses less time and fewer resources (like CPU and memory).

Purpose:

- Makes queries run **faster**
- Reduces **unnecessary data reading**
- Saves **memory and processing power**
- The database system automatically picks the best execution plan

Example: Instead of scanning all 1 million rows, the optimizer uses an index to find the answer in seconds.

Q5. Convert relational algebra $\sigma(A>5)(R)$ into SQL query.

Relational Algebra: $\sigma(A>5)(R)$

This means: Select all rows from relation R where attribute A is greater than 5.

SQL Query:

sql

```
SELECT * FROM R
WHERE A > 5;
```

Q6. Define Checkpointing.

Checkpointing is a technique used in database recovery where the system **saves the current state of the database to disk at regular intervals**.

- When a system crash happens, instead of redoing all transactions from the very beginning, the system **restarts from the last checkpoint**
 - This saves time during recovery
 - It's like a **save point in a video game** — if you fail, you restart from the last save, not from the beginning
-

Q7. Validate the following FDs for relation R(EMP_ID, NAME, SALARY, DEPT, PROJECT):

Given data:

EMP_ID	NAME	SALARY	DEPT	PROJECT
1	A	50000	CS	P1
2	B	60000	EE	P2
3	A	50000	CS	P3
4	C	70000	IT	P2
5	D	80000	EC	P4

FD1: EMP_ID → NAME

Check: Whenever EMP_ID is same, is NAME same?

- EMP_ID=1 → NAME=A (only once) ✓
- EMP_ID=2 → NAME=B ✓
- All EMP_IDs are unique, so no conflict.

✓ FD holds.

FD2: DEPT → PROJECT

Check: Whenever DEPT is same, is PROJECT same?

- DEPT=CS → EMP_ID 1 has P1, EMP_ID 3 has P3 → CS maps to both P1 and P3 ✗

✗ FD does NOT hold.

Two types of dependencies:

1. **Full Functional Dependency** — Y depends on the whole key X, not just part of it
2. **Partial Dependency** — Y depends on only a part of a composite key

Q8. Define Multimedia Database with an example.

A **Multimedia Database** is a database that can **store, manage, and retrieve different types of data** like text, images, audio, video, and animations — all in one system.

Unlike regular databases that only handle text and numbers, multimedia databases handle many formats.

Example: A hospital database that stores:

- Patient name and ID (text)
- MRI scan images (image files)
- Heartbeat recordings (audio)
- Surgery videos (video)

All stored and retrieved together in one system.

Q9. Define Temporal and Spatial DBMS.

Temporal DBMS: A database system that stores data **along with time information**. It tracks how data changes over time.

- Example: Employee salary was ₹50,000 from Jan 2020 to Dec 2022, then ₹70,000 from Jan 2023 onwards.
- It answers questions like: *"What was the salary in 2021?"*

Spatial DBMS: A database system that stores and queries **location-based or geographic data** like points, lines, and areas (polygons).

- Example: Finding all hospitals within 5 km of a location
- Used in Google Maps, GPS, GIS systems

Q10. What is Data Independence? Differentiate between Logical and Physical Data Independence.

Data Independence means the ability to **change the database structure at one level without affecting the levels above it**. Application programs should not need to be rewritten when the database structure changes.

Feature	Logical Data Independence	Physical Data Independence
Definition	Ability to change the conceptual schema without changing external/view schema	Ability to change the internal/physical storage without changing conceptual schema
What changes	Table structure (adding/removing columns)	Storage method (file format, indexing)
Impact	User views remain unaffected	Logical structure remains unaffected

Feature	Logical Data Independence	Physical Data Independence
Harder or easier	Harder to achieve	Easier to achieve
Example	Adding a new column "EMAIL" to STUDENT table without breaking existing apps	Changing from one hard disk to another without changing the table structure

Q11. Define Entity Relationship (ER) Model.

The ER Model (Entity-Relationship Model) is a **visual way to design a database** by showing the real-world objects and the connections between them.

Three main components:

1. **Entity** — A real-world object (e.g., Student, Book, Employee) — shown as a **rectangle**
2. **Attribute** — Properties of an entity (e.g., Name, Age, ID) — shown as an **oval**
3. **Relationship** — Connection between two entities (e.g., Student *enrolls in* Course) — shown as a **diamond**

Example: In a college database:

- Entities: STUDENT, COURSE
- Relationship: ENROLLS
- Attributes of STUDENT: SID, Name, Age

Q12. Given $R(A,B,C)$ with tuples $\{(1,2,3), (1,3,4)\}$, find:

a) $\sigma(A=1)(R)$

Select all rows where $A = 1$:

A	B	C
1	2	3
1	3	4

Both rows satisfy $A=1$, so both are selected.

b) $\pi(B,C)(R)$

Project (show) only columns B and C:

B	C
2	3
3	4

Both rows are kept since they are different.

Q13. Convert SQL query to Relational Algebra: SELECT * FROM R WHERE A=10

SQL Query:

```
sql
SELECT * FROM R WHERE A = 10;
```

Relational Algebra:

$\sigma(A=10)(R)$

- σ is the **selection operator**
- $A=10$ is the **condition**
- R is the **relation (table)**
- This selects all rows from R where A equals 10

Q14. Define Query Optimization.

Query Optimization is the process by which the **database system automatically finds the best and fastest way to execute a given query.**

When you write a SQL query, there can be many different ways to execute it. The query optimizer:

- Analyzes all possible ways to get the result
- Estimates the cost (time, memory) of each way
- Picks the **cheapest/fastest plan**

Goal: Get the correct result in **minimum time using minimum resources**.

Example: Using an index instead of scanning the entire table.

Q15. Check if relation is in 2NF: $R(A,B,C)$, $A \rightarrow B$, $AB \rightarrow C$

Step 1: Find Candidate Key

- $AB^+ = \{A,B,C\}$ — covers all attributes $\rightarrow AB$ is a Superkey
- $A^+ = \{A,B\}$ — not all attributes
- $B^+ = \{B\}$ — not all attributes
- So **AB is the Candidate Key**

Step 2: Find Prime and Non-Prime Attributes

- Prime attributes (part of CK): A, B
- Non-prime attributes: C

Step 3: Check for Partial Dependency

- $A \rightarrow B$ means A (which is a **proper subset of the key AB**) determines B (a prime attribute)
- For 2NF violation: a **non-prime attribute** must depend on a **part of the key**
- C depends on AB (the full key) $\rightarrow$ No partial dependency for C ✓
- $A \rightarrow B$: B is a prime attribute, so this doesn't violate 2NF

✅ **Relation R is in 2NF** (no non-prime attribute partially depends on the key).

Q16. Validate FDs for $R(\text{BOOK_ID}, \text{TITLE}, \text{AUTHOR}, \text{PRICE}, \text{PUBLISHER})$:

Given data:

BOOK_ID	TITLE	AUTHOR	PRICE	PUBLISHER
1	DBMS	X	500	P1
2	AI	Y	600	P2
3	DBMS	X	500	P1
4	ML	Z	700	P3

BOOK_ID	TITLE	AUTHOR	PRICE	PUBLISHER
5	AI	Y	600	P2

FD1: BOOK_ID → TITLE

All BOOK_IDs are unique (1,2,3,4,5) — each maps to exactly one TITLE.

✅ FD holds.

FD2: TITLE → BOOK_ID

Check: Same TITLE maps to same BOOK_ID?

- TITLE=DBMS → BOOK_ID=1 and BOOK_ID=3 → **Two different BOOK_IDs for same TITLE** ❌

❌ FD does NOT hold.

Q17. Define Recoverability in Transactions.

Recoverability means that if a transaction **fails or is rolled back**, the database can be **restored to a correct consistent state** without losing valid data.

A schedule is **recoverable** if: a transaction T2 that reads data written by T1 does not commit **before** T1 commits.

Example:

- T1 writes value X
- T2 reads that value X written by T1
- If T1 fails and rolls back, T2 must also be rolled back (since it read dirty/invalid data)
- If T2 had already committed before T1, that's **irrecoverable** — a bad situation

Simple rule: Never commit a transaction that read data from an uncommitted transaction.

Q18. Give any 2 examples of Spatial Queries.

Example 1: Find all hospitals within 5 km of a given location

```
sql
```

```
SELECT name FROM Hospitals
WHERE ST_Distance(location, ST_Point(77.59, 12.97)) < 5000;
```

This finds hospitals near a specific GPS coordinate (like Bangalore city center).

Example 2: Find all parks inside city boundary

```
sql

SELECT p.name FROM Parks p, CityBoundary c
WHERE c.city_name = 'Bangalore'
AND ST_Within(p.geom, c.boundary);
```

This checks which parks fall completely inside the city polygon.

Q19. Define Valid Time and Transaction Time.

Valid Time:

- The time period when a **fact is true in the real world**
- It represents **real-world timeline**
- Example: An employee was hired on 1st Jan 2020 and resigned on 31st Dec 2023
- This time period (2020-2023) is the valid time — it reflects reality

Transaction Time:

- The time when **data was actually stored or changed in the database**
- It represents the **database timeline**
- Example: The employee's record was entered into the system on 15th Feb 2020
- This is the transaction time — when the database was updated

Key difference: Valid time = when it happened in real world. Transaction time = when it was recorded in the database.

Q20. Define Three-Schema Architecture. Mention its levels.

Three-Schema Architecture is a framework that separates a database into **three levels**, so that changes at one level don't affect other levels. This supports **data independence**.

Three Levels:

1. External Level (View Level)

- What the **end user** sees
- Different users can have different views of the same data
- Example: Student sees only their own marks; Admin sees all marks

2. Conceptual Level (Logical Level)

- The **overall logical structure** of the entire database
- Describes what data is stored and relationships between them
- Example: Tables like STUDENT(ID, NAME, AGE), COURSE(CID, CNAME)

3. Internal Level (Physical Level)

- How data is **physically stored on disk**
- File formats, indexes, storage structures
- Example: Data stored in B-tree index files on hard disk

Q21. Define Entity Relationship (ER) Model.

(Same as Q11 — repeated question)

The **ER Model** is a **high-level conceptual data model** used to design databases visually before actual implementation.

Components:

- **Entity** — Real-world object with independent existence. Shown as rectangle. Example: STUDENT, EMPLOYEE
- **Attribute** — Property of an entity. Shown as oval. Example: Name, Age, ID
- **Relationship** — Association between entities. Shown as diamond. Example: "works in", "enrolls"

Types of relationships:

- One-to-One (1:1)
- One-to-Many (1:N)
- Many-to-Many (M:N)

Q22. Find the result of $R \times S$ if $R(A) = \{1,2\}$, $S(B) = \{x,y\}$

Cartesian Product ($\times$) combines every row of R with every row of S.

R has 2 rows, S has 2 rows → Result has $2 \times 2 = 4$ rows

A	B
1	x
1	y
2	x
2	y

Every value of A is paired with every value of B.

Q23. Schema: Student(student_id, name, marks). Write SQL query to find students with marks > 80.

```
sql

SELECT *
FROM Student
WHERE marks > 80;
```

This query:

- Selects **all columns** (student_id, name, marks)
- From the **Student table**
- Only for students whose **marks are greater than 80**

Q24. Define Dynamic SQL.

Dynamic SQL is SQL code that is **built and executed at runtime** (while the program is running), rather than being fixed/written in advance.

- In **Static SQL**, the query is written and fixed before execution
- In **Dynamic SQL**, the query is **constructed as a string during execution** and then run

Example use case: A search form where the user can search by name, age, or department — depending on what the user enters, different SQL queries are built dynamically.

sql

```
EXECUTE IMMEDIATE 'SELECT * FROM ' || table_name;
```

Here the table name is decided at runtime, not fixed in advance.

Advantage: Very flexible — can handle varying conditions. **Disadvantage:** Harder to debug, slightly slower.

Q25. Check if decomposition is lossless: $R(A,B,C) \rightarrow R_1(A,B), R_2(B,C)$

Lossless decomposition means when we join R_1 and R_2 back together, we get exactly the original relation R — no extra (spurious) tuples and no lost data.

Rule: A decomposition of R into R_1 and R_2 is lossless if the **common attribute(s) between R_1 and R_2 is a key (candidate key) in at least one of them.**

Common attribute between $R_1(A,B)$ and $R_2(B,C)$ = B

Check if B is a key in R_1 or R_2 :

- If $B \rightarrow A$ holds, then B is a key in $R_1(A,B) \rightarrow$ Lossless ✓
- If $B \rightarrow C$ holds, then B is a key in $R_2(B,C) \rightarrow$ Lossless ✓

If B is a candidate key in either R_1 or R_2 , the decomposition is lossless.

Without specific FDs, the condition is: **B must functionally determine the other attribute in at least one of the sub-relations.**

Q26. Validate FDs for $R(\text{STU_ID}, \text{NAME}, \text{MARKS}, \text{DEPT}, \text{COURSE})$:

Given data:

STU_ID	NAME	MARKS	DEPT	COURSE
1	A	78	CS	C1
2	B	60	EE	C1
3	A	78	CS	C2
4	B	60	EE	C3

STU_ID	NAME	MARKS	DEPT	COURSE
5	C	80	IT	C3
6	D	80	EC	C2

FD1: STU_ID → NAME

Check: Same STU_ID always gives same NAME?

- All STU_IDs (1-6) are unique → each maps to exactly one NAME ✓

✅ FD holds.

FD2: DEPT → COURSE

Check: Same DEPT always gives same COURSE?

- DEPT=CS → STU_ID 1 has C1, STU_ID 3 has C2 → CS maps to both C1 and C2 ✗

✗ FD does NOT hold.

Q27. Define Statistical Database.

A **Statistical Database** is a database designed to **store large amounts of numerical data** and support **aggregate queries** (like average, sum, count) while **protecting the privacy of individual records**.

Key features:

- Stores data like census figures, survey results, financial statistics
- Supports: SUM, COUNT, AVG, MIN, MAX
- Must prevent users from figuring out individual data from aggregate results
- Uses privacy techniques like **data perturbation** (adding small random values) and **query size restrictions**

Example: A government census database that can tell you the *average income of a city*, but won't reveal *one specific person's income*.

Q28. Give any 2 examples of Temporal Queries.

Example 1: Valid Time Query — Find salary of employee 101 on a specific date

sql

```
SELECT salary FROM Employee
WHERE emp_id = 101
AND '2021-06-01' BETWEEN vt_start AND vt_end;
```

This finds what the salary was *in reality* on June 1, 2021.

Example 2: Transaction Time Query — Find record as it existed in the database on a past date

sql

```
SELECT * FROM Employee
FOR SYSTEM_TIME AS OF '2023-01-01'
WHERE emp_id = 101;
```

This finds what the database *recorded* for employee 101 on Jan 1, 2023 — useful for auditing.

Q29. List operations performed in OLAP.

OLAP (Online Analytical Processing) supports multi-dimensional analysis of data. The main operations are:

1. **Roll-Up** — Goes from detailed data to summary. Example: City sales → State sales → Country sales (zooming out)
2. **Drill-Down** — Goes from summary to detailed data. Example: Annual sales → Monthly sales → Daily sales (zooming in)
3. **Slice** — Selects **one value of one dimension** to get a smaller view. Example: Show sales data only for Year = 2023
4. **Dice** — Selects **multiple dimensions** to get a sub-cube. Example: Sales for Year=2023 AND Region=South India
5. **Pivot (Rotate)** — **Rotates the view** of data by swapping rows and columns. Example: Change the view from "products in rows, years in columns" to "years in rows, products in columns"

All 29 Part-A questions answered. Ready for Part-B (10-mark questions) whenever you share them!

v. A department controls the project 7. Given 2 tables EMPLOYEE_SAMP (E1, NAME) EMPLOYEE_SAMP (E1, NAME) <table> <tr><th>EMP_ID</th><th>NAME</th></tr> <tr><td>201</td><td>John</td></tr> <tr><td>202</td><td>John</td></tr> <tr><td>203</td><td>John</td></tr> <tr><td>204</td><td>John</td></tr> <tr><td>205</td><td>John</td></tr> <tr><td>206</td><td>John</td></tr> <tr><td>207</td><td>John</td></tr> </table> EMPLOYEE_SAMP (E1, NAME)	EMP_ID	NAME	201	John	202	John	203	John	204	John	205	John	206	John	207	John	1. Find the name of the teacher who teaches KIDMAS 2. Find the name of the teacher who teaches both Computer Organization and Graph Theory 3. Find all teachers taught by Teacher with ID 1001 4. Remove the relation College as STUDENT, TEACHER and its attributes TEACHERID as TID 5. Explain different types of Triggers with examples in DBMS 6. Create a trigger LOAN_ACCOUNT CREATE TRIGGER LOAN_ACCOUNT ON LOAN_ACCOUNT AFTER INSERT, UPDATE, DELETE WHEN (NEW.PAID_AMOUNT > OLD.PAID_AMOUNT) 7. Create triggers for: 8. Prevent payment (PAID_AMOUNT) becomes greater than LOAN_AMOUNT 9. Whenever a payment is made (PAID_AMOUNT) updated, record the details in PAYMENT_LOG 10. Describe different types of functional dependencies with an example 11. Compute the closure of the following set F of functional dependencies for schema Schema R = {A, B, C, D, E}
EMP_ID	NAME																
201	John																
202	John																
203	John																
204	John																
205	John																
206	John																
207	John																

now from Part-B (Q1 - Q13)

PART-B SOLUTIONS (10 Marks Each)

Q1. Describe different types of Database Users with examples.

Database users are people who interact with the database in different ways. They are divided into two main groups:

GROUP 1: Actors on the Scene (People who use the database)

1. Database Administrator (DBA) The DBA is the most powerful user. They manage and control the entire database system.

Responsibilities:

- Creates and maintains the database
- Gives permission to other users (who can access what)
- Takes backups and handles recovery
- Monitors performance and fixes problems
- Defines security rules

Example: In a university, the DBA creates the student database, decides who can add or delete records, and takes daily backups.

2. Database Designers They design the structure (schema) of the database before it is built.

Responsibilities:

- Identify what data needs to be stored
- Design tables, relationships, and constraints
- Create ER diagrams and convert them to relational schemas

Example: A designer creates the STUDENT, COURSE, and ENROLLMENT tables with proper relationships for a college database.

3. End Users These are the people who actually use the database for day-to-day work. They are further divided into:

a) Casual End Users

- Use the database occasionally and need different information each time
- They know how to write some queries
- Example: A manager who sometimes queries the database to check monthly sales reports

b) Naive or Parametric End Users

- Use the database regularly but always do the same type of operations
- They use pre-built forms or applications — they don't write SQL
- Example: Bank tellers who enter customer deposits and withdrawals using a banking application

c) Sophisticated End Users

- Engineers, scientists, or analysts who write complex queries
- They understand SQL well and can do advanced analysis
- Example: A data analyst writing complex SQL to analyze customer purchase patterns

d) Stand-alone Users

- Maintain personal databases using ready-made software
 - Example: A person maintaining their home budget using MS Access
-

GROUP 2: Workers Behind the Scene (People who build and maintain the DBMS)

4. DBMS System Designers and Implementers

- People who design and build the DBMS software itself

- Example: Engineers at Oracle or MySQL who built those database systems

5. Tool Developers

- Develop software tools that work with DBMS
- Example: Developers who build query editors, reporting tools, backup tools

6. Operators and Maintenance Personnel

- People who run and maintain the hardware and software environment
- Example: System administrators who manage servers where the database runs

Summary Table

User Type	Role	Example
DBA	Manages entire database	University DB admin
Designer	Designs structure	Creates ER diagrams
Casual User	Occasional queries	Manager checking reports
Naive User	Regular routine tasks	Bank teller
Sophisticated User	Complex queries	Data analyst
Stand-alone User	Personal database	Home budget tracker
System Implementer	Builds DBMS software	Oracle engineers
Tool Developer	Builds DB tools	Query editor developers

Q2. Construct an ER Diagram for Employee Database with given constraints.

Constraints given:

- i. An employee works for a department
 - ii. Every department is headed by a manager
 - iii. An employee works on one or more projects
 - iv. An employee has dependents
 - v. A department controls the project
-

Entities and their Attributes:

EMPLOYEE: EMP_ID (key), NAME, ADDRESS, SALARY, DOB
DEPARTMENT: DEPT_ID (key), DEPT_NAME, LOCATION
PROJECT: PROJ_ID (key), PROJ_NAME, LOCATION
DEPENDENT: DEPENDENT_NAME, RELATIONSHIP, DOB

Relationships and Cardinality:

Relationship	Entities Involved	Cardinality
WORKS_FOR	EMPLOYEE — DEPARTMENT	Many employees work for one department (N:1)
MANAGES	EMPLOYEE — DEPARTMENT	One employee manages one department (1:1)
WORKS_ON	EMPLOYEE — PROJECT	Many employees work on many projects (M:N)
HAS_DEPENDENT	EMPLOYEE — DEPENDENT	One employee has many dependents (1:N)
CONTROLS	DEPARTMENT — PROJECT	One department controls many projects (1:N)

ER Diagram Description (text form):

```
[EMPLOYEE] -----(WORKS_FOR)-----> [DEPARTMENT]
      |                                   |
      |                                   |
(WORKS_ON)                               (MANAGES)
      |                                   |
[PROJECT]                               [EMPLOYEE]

[EMPLOYEE] -----(HAS)-----> [DEPENDENT]

[DEPARTMENT] --(CONTROLS)-----> [PROJECT]
```

Attributes:

- EMPLOYEE: EMP_ID (underlined/key), NAME, SALARY, ADDRESS, DOB
- DEPARTMENT: DEPT_ID (key), DEPT_NAME
- PROJECT: PROJ_ID (key), PROJ_NAME, LOCATION

- **DEPENDENT:** DEPENDENT_NAME (partial key - double underline),
RELATIONSHIP

Special notes:

- **DEPENDENT** is a **Weak Entity** (shown with double rectangle) — it cannot exist without **EMPLOYEE**
- **HAS_DEPENDENT** is an **Identifying Relationship** (shown with double diamond)
- **WORKS_ON** has attribute: **HOURS** (time employee spends on project)
- **MANAGES** has attribute: **START_DATE**

Participation Constraints:

- Every employee must work for exactly one department — **Total participation** of **EMPLOYEE** in **WORKS_FOR** (double line)
- Department may or may not have a manager — **Partial participation** of **DEPARTMENT** in **MANAGES**

Q3. Write output for Set Operations on **EMPLOYEE_A** and **EMPLOYEE_B**.

Given:

EMPLOYEE_A:

EMP_ID	NAME
201	Amit
202	Sneha
203	Raj
204	Pooja
205	Vikram

EMPLOYEE_B:

EMP_ID	NAME
203	Raj
204	Pooja
206	Neeraj
207	Kavya
208	Rohan

i) $\text{EMPLOYEE_A} \cup \text{EMPLOYEE_B}$ (UNION — All rows from both, no duplicates)

EMP_ID	NAME
201	Amit
202	Sneha
203	Raj
204	Pooja
205	Vikram
206	Neeraj
207	Kavya
208	Rohan

All 8 unique records. Raj and Pooja appear only once even though they exist in both tables.

ii) $\text{EMPLOYEE_A} \cap \text{EMPLOYEE_B}$ (INTERSECTION — Only rows in BOTH tables)

EMP_ID	NAME
203	Raj
204	Pooja

iii) **EMPLOYEE_A** ⋈ **EMPLOYEE_B** (NATURAL JOIN — Join on common attributes)

Both tables have EMP_ID and NAME as common attributes.

Natural join matches rows where EMP_ID AND NAME are same in both.

EMP_ID	NAME
203	Raj
204	Pooja

Same result as intersection here since the matching columns are EMP_ID and NAME.

iv) **EMPLOYEE_A** – **EMPLOYEE_B** (SET DIFFERENCE — Rows in A but NOT in B)

EMP_ID	NAME
201	Amit
202	Sneha
205	Vikram

These 3 employees exist in A but not in B.

v) **EMPLOYEE_A** × **EMPLOYEE_B** (CARTESIAN PRODUCT — Every row of A paired with every row of B)

A has 5 rows, B has 5 rows → $5 \times 5 = 25$ rows in result.

Each row of A is combined with every row of B:

A.EMP_ID	A.NAME	B.EMP_ID	B.NAME
201	Amit	203	Raj
201	Amit	204	Pooja

A.EMP_ID	A.NAME	B.EMP_ID	B.NAME
201	Amit	206	Neeraj
201	Amit	207	Kavya
201	Amit	208	Rohan
202	Sneha	203	Raj
202	Sneha	204	Pooja
...	...	...	...

(continues for all 25 combinations)

Q4. Differentiate File Processing Systems with DBMS.

Feature	File Processing System	DBMS
Data Storage	Data stored in separate files for each application	Data stored centrally in one integrated system
Data Redundancy	High redundancy — same data stored in multiple files	Low redundancy — data stored once, shared by all
Data Inconsistency	High — updating one file may not update others	Low — one update reflects everywhere
Data Sharing	Difficult — different programs use different file formats	Easy — all applications access the same database
Data Independence	No data independence — changing file structure breaks programs	Supports both logical and physical data independence
Security	Difficult to implement security — anyone can open files	Fine-grained security — control who sees what
Concurrency	No built-in support for multiple users at same time	Full concurrency control — multiple users simultaneously
Crash Recovery	No recovery mechanism — data may be lost on crash	Built-in backup and recovery mechanisms

Feature	File Processing System	DBMS
Query Language	No standard query language — must write programs	Standard SQL for querying
Data Integrity	Hard to enforce constraints	Constraints like PRIMARY KEY, NOT NULL enforced automatically
Cost	Low initial cost	Higher initial cost but saves long-term
Example	Old library card catalog in text files	MySQL, Oracle managing bank records

Advantages of DBMS over File Systems:

1. Eliminates data redundancy
2. Ensures data consistency
3. Provides data security
4. Supports multiple users simultaneously
5. Easy to query with SQL
6. Automatic backup and recovery
7. Enforces data integrity

Disadvantages of File Processing (Problems it caused):

1. Data redundancy wastes storage
2. Data inconsistency when same data stored in multiple files
3. Data isolation — data scattered, hard to find
4. Atomicity problems — partial updates cause corruption
5. Security is hard to enforce
6. No concurrent access support

Q5. Explain Advantages and Disadvantages of Triggers in DBMS.

A **Trigger** is a set of SQL statements that **automatically execute** when a specific event (INSERT, UPDATE, DELETE) happens on a table.

Advantages of Triggers:

2. Data Integrity Maintenance Triggers help maintain correct and valid data in the database. Example: A trigger can prevent inserting a salary below minimum wage.

3. Enforcing Business Rules Complex business rules that cannot be handled by simple constraints can be enforced using triggers. Example: "An employee's bonus cannot exceed 50% of their salary" — this rule can be enforced by a trigger.

4. Auditing and Logging Triggers can automatically record who changed what and when — useful for tracking changes. Example: Every time an employee's salary is updated, a trigger logs the old salary, new salary, and timestamp in an audit table.

5. Scheduled Task Automation Triggers eliminate the need to manually run certain tasks — they run automatically when needed. Example: When an order is placed, a trigger automatically updates the inventory count.

6. Referential Integrity Triggers can help maintain relationships between tables beyond what foreign keys alone can do. Example: When a department is deleted, a trigger can reassign employees to another department instead of just blocking the deletion.

Disadvantages of Triggers:

1. Difficult to Debug When a trigger causes an error, it is very hard to find where the problem is because triggers run automatically and invisibly.

2. Performance Overhead Every INSERT/UPDATE/DELETE operation also fires the trigger, which adds extra processing time. This slows down the database, especially with complex triggers.

3. Not Compiled (in some DBMS) Triggers are interpreted at runtime, not compiled. This makes them slower than stored procedures.

4. Hidden Logic Business logic hidden inside triggers makes applications harder to understand and maintain. A developer looking at the application code won't see the trigger logic unless they check the database.

5. Cascading Triggers One trigger can fire another trigger which fires another — this chain can cause unexpected behavior and is very hard to control. Example: Trigger A fires Trigger B, which fires Trigger C — this cascading can cause infinite loops.

6. Increased Database Load Heavy use of triggers increases the load on the database server and can reduce overall system performance.

Q6. Relational Algebra Queries on Student-Teacher-College Schema.

Given:

- STUDENT (ROLL_NO, NAME, ADDRESS)
 - TEACHERS (TEACHER_ID, TEACHER_NAME, TEACHING_SUBJECT)
 - COLLEGE (ROLL_NO, TEACHER_ID)
-

i. Find the name of students who live in Delhi.

$\sigma(\text{ADDRESS}='Delhi')(\text{STUDENT})$

Then project only NAME:

$\pi(\text{NAME})(\sigma(\text{ADDRESS}='Delhi')(\text{STUDENT}))$

This selects all students from the STUDENT table where ADDRESS is Delhi, then shows only the NAME column.

ii. Find the name of teacher who teaches RDBMS.

$\pi(\text{TEACHER_NAME})(\sigma(\text{TEACHING_SUBJECT}='RDBMS')(\text{TEACHERS}))$

This selects from TEACHERS table where TEACHING_SUBJECT is RDBMS, then shows the TEACHER_NAME.

iii. Find the name of teacher who teaches both Computer Organization and Graph Theory.

Step 1 — Find teachers who teach Computer Organization:

$T1 = \pi(\text{TEACHER_ID})(\sigma(\text{TEACHING_SUBJECT}='Computer Organization')(\text{TEACHERS}))$

Step 2 — Find teachers who teach Graph Theory:

$T2 = \pi(\text{TEACHER_ID})(\sigma(\text{TEACHING_SUBJECT}='Graph Theory')(\text{TEACHERS}))$

Step 3 — Find teachers who teach BOTH (Intersection):

$T3 = T1 \cap T2$

Step 4 — Get their names:

$\pi(\text{TEACHER_NAME})(T3 \bowtie \text{TEACHERS})$

iv. Find all students taught by Teacher with ID 1001.

Step 1 — Find ROLL_NOs from COLLEGE table where TEACHER_ID = 1001:

$C1 = \pi(\text{ROLL_NO})(\sigma(\text{TEACHER_ID}=1001)(\text{COLLEGE}))$

Step 2 — Join with STUDENT to get student names:

$\pi(\text{NAME})(C1 \bowtie \text{STUDENT})$

v. Rename the relation College as STUDENT_TEACHER and attribute TEACHERID as TID.

$\rho(\text{STUDENT_TEACHER}(\text{ROLL_NO}, \text{TID}))(\text{COLLEGE})$

The ρ (rho) is the rename operator. This renames the relation COLLEGE to STUDENT_TEACHER and renames TEACHER_ID to TID.

Q7. Explain Different Types of Triggers with Examples in DBMS.

A **trigger** is a database object that automatically runs a set of SQL statements when a specific event occurs on a table.

Types of Triggers based on TIMING:

1. BEFORE Trigger Executes **before** the INSERT, UPDATE, or DELETE operation happens. Used to: validate data, modify values before saving

Example: Before inserting a student's marks, check if marks are between 0 and 100:

```
sql

CREATE TRIGGER check_marks
BEFORE INSERT ON Student
FOR EACH ROW
BEGIN
    IF NEW.marks < 0 OR NEW.marks > 100 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Invalid marks!';
    END IF;
END;
```

2. AFTER Trigger Executes **after** the INSERT, UPDATE, or DELETE operation is completed. Used to: log changes, update related tables

Example: After updating salary, record the change in an audit log:

sql

```
CREATE TRIGGER salary_audit
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Audit_Log(emp_id, old_salary, new_salary, change_date)
    VALUES(OLD.emp_id, OLD.salary, NEW.salary, NOW());
END;
```

Types of Triggers based on EVENT:

3. INSERT Trigger (AFTER INSERT / BEFORE INSERT) Fires when a new row is inserted. Example: After a new order is placed, automatically reduce stock count.

4. UPDATE Trigger (AFTER UPDATE / BEFORE UPDATE) Fires when an existing row is modified. Example: After employee salary is updated, log it to audit table.

5. DELETE Trigger (AFTER DELETE / BEFORE DELETE) Fires when a row is deleted. Example: Before deleting a customer, check if they have pending orders.

Types based on LEVEL:

6. Row-Level Trigger (FOR EACH ROW) Fires once for **each row** that is affected. Example: If 5 employees get salary updated, the trigger runs 5 times.

7. Statement-Level Trigger (FOR EACH STATEMENT) Fires once for the **entire SQL statement**, regardless of how many rows are affected. Example: The trigger runs only once even if 100 rows are updated.

Summary Table:

Type	When it fires
BEFORE INSERT	Before new row is added

Type	When it fires
AFTER INSERT	After new row is added
BEFORE UPDATE	Before row is changed
AFTER UPDATE	After row is changed
BEFORE DELETE	Before row is removed
AFTER DELETE	After row is removed

Q8. Create Triggers for LOAN_ACCOUNT and PAYMENT_LOG.

Given tables:

- LOAN_ACCOUNT (LOAN_ID, CUST_ID, LOAN_AMOUNT, PAID_AMOUNT, STATUS)
- PAYMENT_LOG (LOG_ID, LOAN_ID, OLD_PAID, NEW_PAID, PAYMENT_DATE)

i. Trigger to PREVENT payment if PAID_AMOUNT becomes greater than LOAN_AMOUNT:

```
sql

CREATE TRIGGER prevent_overpayment
BEFORE UPDATE ON LOAN_ACCOUNT
FOR EACH ROW
BEGIN
    IF NEW.PAID_AMOUNT > NEW.LOAN_AMOUNT THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Paid amount cannot exceed loan amount!';
    END IF;
END;
```

How it works:

- This trigger fires **BEFORE** any UPDATE on LOAN_ACCOUNT
- It checks if the new PAID_AMOUNT would be more than LOAN_AMOUNT
- If yes, it raises an error and **stops the update from happening**
- NEW.PAID_AMOUNT refers to the value being updated to

ii. Trigger to RECORD payment details in PAYMENT_LOG whenever PAID_AMOUNT is updated:

```
sql

CREATE TRIGGER log_payment
AFTER UPDATE ON LOAN_ACCOUNT
FOR EACH ROW
BEGIN
    IF OLD.PAID_AMOUNT <> NEW.PAID_AMOUNT THEN
        INSERT INTO PAYMENT_LOG
            (LOAN_ID, OLD_PAID, NEW_PAID, PAYMENT_DATE)
        VALUES
            (OLD.LOAN_ID, OLD.PAID_AMOUNT, NEW.PAID_AMOUNT, NOW());
    END IF;
END;
```

How it works:

- This trigger fires **AFTER** an UPDATE on LOAN_ACCOUNT
- It checks if PAID_AMOUNT actually changed
- OLD.PAID_AMOUNT = the value before update
- NEW.PAID_AMOUNT = the value after update
- It inserts a new log record into PAYMENT_LOG with old value, new value, and current date/time

Q9. Describe Different Types of Functional Dependencies with Examples.

A **Functional Dependency (FD)** means: if you know the value of attribute X, you can always find the unique value of attribute Y. Written as $X \rightarrow Y$.

1. Full Functional Dependency

Y is **fully functionally dependent** on X if it depends on the **entire X**, not just a part of it.

Example:

- Relation: WORKS_ON(EMP_ID, PROJ_ID, HOURS)
- $\{EMP_ID, PROJ_ID\} \rightarrow HOURS$

- HOURS depends on both EMP_ID AND PROJ_ID together
 - You can't find HOURS from EMP_ID alone or PROJ_ID alone
 - This is a **Full FD** ✓
-

2. Partial Functional Dependency

Y is **partially dependent** on X if it depends on only a **part (subset)** of the composite key X.

Example:

- Relation: EMP_PROJ(EMP_ID, PROJ_ID, ENAME, PNAME, HOURS)
- {EMP_ID, PROJ_ID} is the key
- $EMP_ID \rightarrow ENAME$ (ENAME depends only on EMP_ID, not the full key)
- This is a **Partial FD** — causes 2NF violation ✓

Fix: Move ENAME to a separate relation with EMP_ID.

3. Transitive Functional Dependency

If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$ is a **transitive dependency**. Z depends on X *indirectly* through Y.

Example:

- EMP_DEPT(EMP_ID, DEPT_ID, DEPT_NAME)
- $EMP_ID \rightarrow DEPT_ID$ (employee belongs to department)
- $DEPT_ID \rightarrow DEPT_NAME$ (department has a name)
- Therefore: $EMP_ID \rightarrow DEPT_NAME$ (transitively)
- DEPT_NAME is **transitively dependent** on EMP_ID through DEPT_ID
- This causes 3NF violation ✓

Fix: Separate DEPT_ID and DEPT_NAME into their own relation.

4. Trivial Functional Dependency

An FD $X \rightarrow Y$ is **trivial** if Y is a **subset of X** (Y is already part of X).

Example:

- $\{EMP_ID, NAME\} \rightarrow EMP_ID$

- EMP_ID is already part of the left side, so this is trivially true
- Trivial FDs are always true and not interesting

5. Non-Trivial Functional Dependency

An FD $X \rightarrow Y$ is **non-trivial** if Y is NOT a subset of X.

Example:

- EMP_ID $\rightarrow$ NAME
- NAME is not part of EMP_ID
- This is a meaningful, non-trivial FD

6. Multi-Valued Dependency (MVD)

$X \twoheadrightarrow Y$ means: for each value of X, there is a **set of values** of Y independent of other attributes.

Example:

- EMPLOYEE(EMP_ID, SKILL, LANGUAGE)
- An employee can have multiple skills AND multiple languages independently
- EMP_ID $\twoheadrightarrow$ SKILL and EMP_ID $\twoheadrightarrow$ LANGUAGE
- This is a Multi-valued dependency — causes 4NF violation

Summary Table:

Type	Definition	Example
Full FD	Y depends on all of X	{EMP,PROJ} $\rightarrow$ HOURS
Partial FD	Y depends on part of X	EMP_ID $\rightarrow$ ENAME in composite key
Transitive FD	$X \rightarrow Y$ and $Y \rightarrow Z$ means $X \rightarrow Z$	EMP $\rightarrow$ DEPT, DEPT $\rightarrow$ DNAME
Trivial FD	Y is subset of X	{A,B} $\rightarrow$ A
Non-Trivial FD	Y is not subset of X	$A \rightarrow B$
Multi-valued FD	X determines a set of Y values	EMP $\twoheadrightarrow$ SKILL

Q10. Compute Closure of FDs for $R=\{A,B,C,D,E\}$, $\{AB \rightarrow C, CD \rightarrow E, DE \rightarrow B\}$. List candidate keys and superkeys.

Given FDs:

- $AB \rightarrow C$
 - $CD \rightarrow E$
 - $DE \rightarrow B$
-

Step 1: Find Closure of various attribute sets

We need to find which combinations can determine ALL attributes $\{A, B, C, D, E\}$.

Try AB:

- Start: $\{A, B\}$
- $AB \rightarrow C$, so add $C \rightarrow \{A, B, C\}$
- No more FDs fire (CD needs D, DE needs D)
- $AB^+ = \{A, B, C\}$ — **Not a superkey** (missing D, E)

Try ABD:

- Start: $\{A, B, D\}$
- $AB \rightarrow C$, add $C \rightarrow \{A, B, C, D\}$
- $CD \rightarrow E$, add $E \rightarrow \{A, B, C, D, E\}$
- $ABD^+ = \{A, B, C, D, E\}$  **Superkey!**
- Can we remove any attribute?
 - BD^+ : BD alone $\rightarrow$ nothing fires $\rightarrow \{B, D\}$ only — Not superkey
 - AD^+ : AD alone $\rightarrow$ nothing fires $\rightarrow \{A, D\}$ only — Not superkey
 - $AB^+ = \{A, B, C\}$ — Not superkey
- No proper subset of ABD is a superkey $\rightarrow$ **ABD is a Candidate Key!**

Try ACD:

- Start: $\{A, C, D\}$
- $CD \rightarrow E$, add $E \rightarrow \{A, C, D, E\}$
- $DE \rightarrow B$, add $B \rightarrow \{A, B, C, D, E\}$
- $ACD^+ = \{A, B, C, D, E\}$  **Superkey!**

- Can we remove any?
 - CD^+ : $CD \rightarrow E \rightarrow \{C,D,E\}$, $DE \rightarrow B \rightarrow \{B,C,D,E\}$ — Not all (missing A)
 - AD^+ : $\{A,D\}$ — nothing fires — Not superkey
 - AC^+ : $\{A,C\}$ — nothing fires — Not superkey
- No proper subset works $\rightarrow$ **ACD is a Candidate Key!**

Try ADE:

- Start: $\{A, D, E\}$
- $DE \rightarrow B$, add $B \rightarrow \{A, B, D, E\}$
- $AB \rightarrow C$, add $C \rightarrow \{A, B, C, D, E\}$
- $ADE^+ = \{A, B, C, D, E\}$  **Superkey!**
- Can we remove any?
 - DE^+ : $DE \rightarrow B \rightarrow \{B,D,E\}$, no more $\rightarrow$ Not superkey (missing A,C)
 - AE^+ : $\{A,E\}$ — nothing fires $\rightarrow$ Not superkey
 - AD^+ : $\{A,D\}$ — Not superkey
- **ADE is a Candidate Key!**

i) Candidate Keys for R:

- **ABD**
- **ACD**
- **ADE**

ii) Some Superkeys:

(Superkeys = candidate keys + any additional attributes)

- ABD, ABCD, ABDE, ABCDE
- ACD, ACDE, ABCD
- ADE, ABDE, ACDE, ABCDE

Q11. Illustrate Testing for Conflict Serializability with an Algorithm.

What is Serializability?

When multiple transactions run at the same time (concurrently), the final result should be the same as if they ran one after another (serially). This is called **serializability**.

What is Conflict?

Two operations **conflict** if:

1. They belong to **different transactions**
2. They access the **same data item**
3. At least **one of them is a WRITE**

Conflicting pairs: Read-Write, Write-Read, Write-Write

Algorithm: Precedence Graph (Serialization Graph) Method

Steps:

1. Create a node for each transaction (T1, T2, T3...)
 2. For each conflict between two operations:
 - If Ti does an operation BEFORE Tj on same data, draw arrow: **Ti → Tj**
 3. If the graph has **NO cycle** → Schedule is **Conflict Serializable** ✓
 4. If the graph has **a cycle** → Schedule is **NOT Conflict Serializable** ✗
-

Example:

Schedule:

T1: R(A), W(A)
T2: R(A), W(A)

Step	T1	T2
1	R(A)	
2		R(A)
3	W(A)	
4		W(A)

Find Conflicts:

- T1 W(A) at step 3, T2 R(A) at step 2 — T2 reads BEFORE T1 writes → T2 → T1
- T1 R(A) at step 1, T2 W(A) at step 4 — T1 reads BEFORE T2 writes → T1 → T2

Precedence Graph:

- T1 → T2 (T1 reads A before T2 writes A)
- T2 → T1 (T2 reads A before T1 writes A)

Result: T1 → T2 → T1 → **CYCLE EXISTS!**

Conclusion: This schedule is **NOT conflict serializable**.

Non-conflicting Example:

If we only had T1 → T2 with no arrow back, there's no cycle → **Conflict Serializable** ✓
(equivalent to serial schedule T1 followed by T2).

Q12. Explain Desirable Properties of Transactions in DBMS (ACID Properties).

The four desirable properties of transactions are called **ACID properties**. These ensure that database transactions are processed reliably.

A — Atomicity ("All or Nothing")

A transaction is treated as a **single unit**. Either ALL operations in it complete successfully, or NONE of them take effect.

Example: Bank transfer of ₹5000 from Account A to Account B:

- Step 1: Deduct ₹5000 from A
- Step 2: Add ₹5000 to B

If the system crashes after Step 1 but before Step 2, Atomicity ensures that Step 1 is also **rolled back** — A doesn't lose money without B receiving it.

How achieved: ROLLBACK and COMMIT commands; Undo logs.

C — Consistency

A transaction must take the database from one **valid/consistent state** to another **valid/consistent state**. All data rules and constraints must hold before and after the transaction.

Example: If a bank rule says "account balance cannot go below ₹0", then any transaction that tries to withdraw more money than available will be rejected — consistency is maintained.

How achieved: Integrity constraints (PRIMARY KEY, FOREIGN KEY, CHECK, etc.)

I — Isolation

Multiple transactions executing at the same time should not **interfere with each other**. Each transaction should feel like it is the only one running.

Example: T1 is reading account balance while T2 is updating it. T1 should not see T2's partial update — it should see either the old balance or the fully updated balance, not something in between.

How achieved: Concurrency control mechanisms like locking, timestamp ordering.

D — Durability

Once a transaction is **committed (successfully completed)**, its changes must be **permanent** — even if the system crashes right after.

Example: You transfer money and the system confirms success. If the server crashes 1 second later, the transfer should still be there when the system restarts.

How achieved: Write-ahead logging (WAL), database backups, checkpoints.

Summary Table:

Property	Meaning	Simple way to remember
Atomicity	All or nothing	A transaction can't be half-done
Consistency	Valid state to valid state	Rules always satisfied
Isolation	Transactions don't interfere	Each transaction runs alone
Durability	Committed = permanent	Changes survive crashes

Q13. Explain NoSQL Databases and Types with Examples.

What is NoSQL?

NoSQL stands for "Not Only SQL". These are databases that are **non-relational** — they don't use the traditional table-based structure of SQL databases.

Why NoSQL?

- Traditional SQL databases struggle with very large amounts of data (Big Data)
- Modern applications need flexible data structures that change frequently
- NoSQL databases can easily scale across many servers (horizontal scaling)

Key Differences: SQL vs NoSQL

Feature	SQL	NoSQL
Data model	Tables with fixed rows/columns	Flexible: documents, key-value, graphs
Schema	Fixed, must be defined upfront	Dynamic, can change anytime
Scaling	Vertical (bigger server)	Horizontal (more servers)
Best for	Structured data, banking, ERP	Big data, social media, IoT

Types of NoSQL Databases:

1. Key-Value Store

Data is stored as simple **key** → **value** pairs. Like a dictionary or lookup table.

- Fastest type of NoSQL database
- Best for: Caching, session management, shopping carts
- Examples: **Redis, DynamoDB, Riak**

Data example:

```
"user_101" → {"name": "Raj", "age": 22}
"cart_101" → ["laptop", "mouse", "keyboard"]
```

2. Document Store

Data is stored as **documents** (usually in JSON or XML format). Each document can have different fields — very flexible.

- Best for: Content management, user profiles, product catalogs
- Examples: **MongoDB, CouchDB, Firestore**

Data example (one document in MongoDB):

```
json
{
  "_id": "P001",
  "name": "Laptop",
  "brand": "Dell",
  "price": 55000,
  "specs": {"RAM": "16GB", "Storage": "512GB"}
}
```

Each product document can have different fields — no rigid structure required.

3. Column-Family Store (Wide-Column Store)

Data is stored in **rows and columns** but unlike SQL, different rows can have **different columns**. Data for one row is stored together.

- Best for: Time-series data, IoT sensor data, analytics
- Examples: **Apache Cassandra, HBase**

Data example:

```
Row: User001 → {name: "Raj", city: "Bangalore", age: 25}
Row: User002 → {name: "Sneha", email: "sneha@mail.com"}
```

(Notice User002 has different columns than User001 — this is allowed!)

4. Graph Database

Data is stored as **nodes** (entities) and **edges** (relationships between them). Best when relationships between data are important.

- Best for: Social networks, fraud detection, recommendation engines
- Examples: **Neo4j**, **Amazon Neptune**

Data example:

```
(Alice) --[FRIEND]--> (Bob)
(Bob) --[LIKES]--> (Post123)
(Alice) --[BOUGHT]--> (Laptop)
```

Query example: "Find all friends of Alice who also bought a Laptop" — very natural in graph DB, very complex in SQL.

Advantages of NoSQL:

1. Handles huge amounts of data (Big Data)
2. Very fast read/write operations
3. Flexible schema — no need to pre-define all columns
4. Easily scales horizontally (add more servers)
5. Stores complex, nested data naturally (like JSON)

Disadvantages of NoSQL:

1. No standard query language (each DB has its own)
 2. Weaker consistency guarantees (eventual consistency)
 3. Not ideal for complex joins and relationships
 4. Less mature than SQL databases
 5. Not suitable for all types of applications
-

All 13 Part-B questions (Q1 to Q13) answered completely! Ready for Q14 onwards whenever you share them.

13. Convert SQL query to relational algebra: SELECT * FROM R WHE

PASTED

before generating answer attach question along with answer as you did for earlier questions. so that if you misunderstood question I can correct you.

PART-B SOLUTIONS (Q14 to Q38) — 10 Marks Each

Q14. Explain the Architecture of Data Warehousing.

A Data Warehouse is a central system that collects data from many different sources, cleans it, organizes it, and makes it available for analysis and reporting.

Think of it like a big library — books (data) come from many places, get organized on shelves (structured), and people can easily find and read them (query and analyze).

Complete Architecture (Layer by Layer):

LAYER 1: Data Sources

These are the original places where data is created and stored. They feed data into the warehouse.

Types of sources:

- **OLTP Databases** — Operational databases like sales, inventory, HR systems (MySQL, Oracle)
- **Flat Files** — Excel sheets, CSV files, text files
- **External Sources** — Government data, market data, social media feeds
- **ERP/CRM Systems** — SAP, Salesforce, etc.
- **Web Logs** — Website click data, user activity

Problem: All these sources have different formats, different structures, and may have duplicate or incorrect data. That's why we need the ETL layer.

LAYER 2: ETL Layer (Extract, Transform, Load)

This is the most important processing layer. It acts like a filter/cleaner.

Extract:

- Pull raw data from all different source systems
- Data may come in different formats (CSV, JSON, SQL tables)

Transform:

- Clean the data (remove duplicates, fix errors, fill missing values)
- Convert formats (e.g., date formats from DD/MM/YYYY to YYYY-MM-DD)
- Standardize values (e.g., "Male" and "M" both become "M")
- Combine data from multiple sources into one unified format
- Apply business rules

Load:

- Load the cleaned, transformed data into the Data Warehouse
- Can be full load (load everything) or incremental load (load only new/changed data)

Example: Sales data from 5 different regional offices in different formats → ETL cleans and combines them → Loads into warehouse as one unified sales table.

LAYER 3: Data Warehouse Storage (Central Repository)

This is where all the cleaned, integrated data is stored. It is:

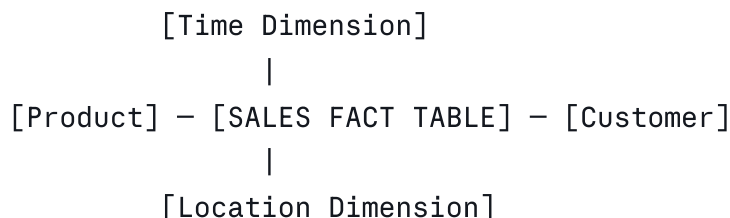
- **Subject-Oriented** — Organized around subjects like Sales, Finance, HR (not around applications)
- **Integrated** — Data from all sources combined into one consistent format
- **Time-Variant** — Stores historical data, not just current data (5-10 years of history)
- **Non-Volatile** — Once data is loaded, it is not changed or deleted (read-only)

Schema Types used:

- **Star Schema** — One central Fact table surrounded by Dimension tables (like a star shape). Simple and fast for queries.

- **Snowflake Schema** — Like Star Schema but dimension tables are further normalized (broken into sub-tables). More complex but saves space.

Example of Star Schema:



The Fact Table contains measurable data (sales amount, quantity) and the Dimension tables contain descriptive data (product name, customer name, date).

LAYER 4: Metadata Repository

Metadata means "data about data." This layer stores:

- What data exists in the warehouse
- Where it came from
- When it was last updated
- What transformations were applied
- Data definitions and business rules

It helps users understand the data and helps administrators manage the warehouse.

LAYER 5: Data Marts

A Data Mart is a **smaller, focused version of the data warehouse** for a specific department or subject area.

- Sales Mart — Only sales-related data for the sales team
- Finance Mart — Financial data for the finance department
- HR Mart — Employee data for HR department

Why Data Marts?

- Faster queries (less data to search through)
- Department-specific data only
- Better performance for specific users

LAYER 6: OLAP Server

OLAP (Online Analytical Processing) server allows multi-dimensional analysis.

Types:

- **ROLAP** (Relational OLAP) — Works on top of relational databases
- **MOLAP** (Multidimensional OLAP) — Data stored in multidimensional cubes
- **HOLAP** (Hybrid OLAP) — Combination of both

Supports operations: Roll-up, Drill-down, Slice, Dice, Pivot

LAYER 7: Front-End / Presentation Layer

This is what business users actually see and use:

- **Dashboards** — Visual displays of key metrics (Tableau, Power BI)
 - **Reports** — Formatted printable reports
 - **Ad-hoc Queries** — Users write their own custom queries
 - **Data Mining** — Discovering hidden patterns using algorithms (clustering, classification)
 - **Spreadsheet Tools** — Excel connected to the warehouse
-

Complete Architecture Diagram (Text Form):

```
[Source Systems]
  OLTP DBs | Files | ERP | Web Logs
      ↓
    [ETL Layer]
  Extract → Transform → Load
      ↓
[Data Warehouse Storage]
  Star/Snowflake Schema
  + Metadata Repository
      ↓
    [Data Marts]
  Sales | Finance | HR
      ↓
```

[OLAP Server]
ROLAP | MOLAP | HOLAP



[Presentation / Access Layer]
Dashboards | Reports | Mining

Key Characteristics of Data Warehouse (4 properties):

Property	Meaning	Example
Subject-Oriented	Organized by subject, not application	Sales data, not Sales Application
Integrated	Data from all sources in one format	Date stored as YYYY-MM-DD everywhere
Time-Variant	Stores history	5 years of sales data
Non-Volatile	Data doesn't change once loaded	Old records stay as they are

Q15. Explain Object-Oriented Database Management System (OODBMS).

OODBMS is a database system that combines the features of Object-Oriented Programming (OOP) with database capabilities. It stores data as objects — just like objects in programming languages like Java or C++.

In traditional (relational) databases, data is stored in tables. In OODBMS, data is stored as objects that have both data (attributes) and behavior (methods/functions) together.

Core Concepts of OODBMS:

1. Object The basic unit of OODBMS. An object contains:

- **Attributes** (data) — the properties of the object
- **Methods** (behavior) — the functions the object can perform

Example:

Object: Student
Attributes: student_id = 101, name = "Raj", age = 20
Methods: get_grade(), calculate_gpa(), enroll_course()

2. Class A class is a **blueprint/template** for creating objects. All objects of the same class share the same structure.

Example:

```
Class: VEHICLE
  Attributes: make, model, year, color
  Methods: start(), stop(), accelerate()
```

From this class, we can create objects like Car, Truck, Bike.

3. Inheritance A subclass (child class) can **inherit** all attributes and methods from its parent class and can also add its own.

Example:

```
Parent Class: VEHICLE
  ↓
Child Class: CAR (inherits make, model, year + adds numDoors)
Child Class: TRUCK (inherits make, model, year + adds payloadCapacity)
```

This avoids rewriting the same code again.

4. Encapsulation The internal data of an object is **hidden** from the outside. It can only be accessed through the object's methods (like getter and setter functions).

Example: You can't directly change a student's GPA — you must call the `update_gpa()` method which applies proper rules.

This protects data from being incorrectly modified.

5. Polymorphism The same method name can **behave differently** for different objects.

Example:

```
Method: calculate_area()
  For Circle object  $\rightarrow \pi \times r^2$ 
```


For Rectangle object $\rightarrow \text{length} \times \text{width}$
For Triangle object $\rightarrow \frac{1}{2} \times \text{base} \times \text{height}$

Same method name, different behavior depending on the object.

6. Object Identity (OID) Every object in the database gets a **unique system-generated identifier** called OID.

- Unlike relational databases where you need a primary key, OID is automatic
 - Two objects can have the same data but still be different objects (different OIDs)
-

7. Persistence Objects created in a program normally disappear when the program ends. In OODBMS, objects can be made **persistent** — they are saved to the database and exist even after the program closes.

8. Complex Types OODBMS supports complex, nested data structures:

- An object can contain another object
- Collections (lists, sets) of objects
- References to other objects

Example: A **DEPARTMENT** object can contain a list of **EMPLOYEE** objects.

OQL (Object Query Language)

OODBMS uses OQL for querying, which is similar to SQL but works on objects.

Example:

```
sql
SELECT v FROM Vehicles v WHERE v.make = 'Toyota'
```

Advantages of OODBMS:

Advantage	Explanation
Natural modeling	Real-world objects modeled directly — no mismatch
Handles complex data	Nested objects, multimedia, documents
Code reuse	Inheritance allows reusing existing class definitions
Better performance	No joins needed — objects directly reference other objects
Supports multimedia	Images, audio, video stored as objects naturally

Disadvantages of OODBMS:

Disadvantage	Explanation
Not widely adopted	SQL/Relational databases dominate industry
Complex design	Object design requires OOP expertise
No standard query language	Different systems use different query languages
Less mature	Not as developed or tested as relational databases
Poor for simple data	Overhead not justified for simple, flat data

Examples of OODBMS:

- **db4o** (database for objects)
- **ObjectDB**
- **Versant**
- **GemStone/S**

OODBMS vs RDBMS:

Feature	RDBMS	OODBMS
Data stored as	Tables (rows/columns)	Objects
Relationships	Foreign keys and joins	Direct object references

Feature	RDBMS	OODBMS
Complex data	Difficult (multiple tables)	Natural (nested objects)
Query language	SQL	OQL
Best for	Business data, transactions	CAD, multimedia, scientific data

Q16. Describe the Structure of a Deductive Database System with an Example.

A **Deductive Database** combines a regular relational database with logic programming. It can derive new facts from existing facts using rules — like a reasoning engine.

Regular databases only store facts you explicitly put in. A Deductive Database can figure out new facts on its own using rules.

Simple analogy: If you know "Raj is a parent of Priya" and "Priya is a parent of Arjun", a deductive database can automatically figure out "Raj is a grandparent of Arjun" — without you explicitly storing that.

Structure of a Deductive Database System:

1. EDB — Extensional Database (Base Facts) These are the **explicitly stored facts** — the actual data in the database tables. These are facts entered by the user.

Example:

```
parent(tom, bob).    -- tom is parent of bob
parent(bob, ann).    -- bob is parent of ann
parent(ann, pat).    -- ann is parent of pat
```

These are the base facts that are directly stored.

2. IDB — Intensional Database (Derived Facts + Rules) These are **rules** written in logic that allow the system to **derive new facts** from the EDB.

Rules are written in the form: **Head :- Body** (Read as: "Head is TRUE if Body is TRUE")

Example rules:

```
ancestor(X, Y) :- parent(X, Y).  
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```

Rule 1: X is an ancestor of Y if X is a direct parent of Y.

Rule 2: X is an ancestor of Y if X is a parent of Z and Z is an ancestor of Y (recursive rule).

3. Rule Engine / Inference Engine This is the component that **applies the rules** to the base facts and **derives new conclusions**.

It repeatedly applies rules until no new facts can be derived. This process is called **bottom-up evaluation** or **forward chaining**.

4. Query Processor Accepts queries from users and answers them using both the stored facts (EDB) and derived facts (IDB).

5. Datalog — The Language Used Deductive databases use **Datalog**, which is a logic programming language (a subset of Prolog). It is more powerful than SQL for reasoning and recursive queries.

Complete Example — Ancestor Derivation:

EDB (Base Facts stored in database):

```
parent(tom, bob).  
parent(bob, ann).  
parent(ann, pat).
```

IDB (Rules):

```
ancestor(X, Y) :- parent(X, Y).  
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```

User Query:

```
?- ancestor(tom, pat).
```

How the system reasons:

- tom is parent of bob (direct) $\rightarrow$ ancestor(tom, bob) ✓
- bob is parent of ann $\rightarrow$ ancestor(bob, ann) ✓
- tom is parent of bob AND bob is ancestor of ann $\rightarrow$ ancestor(tom, ann) ✓
- ann is parent of pat $\rightarrow$ ancestor(ann, pat) ✓
- tom is ancestor of ann AND ann is parent of pat $\rightarrow$ ancestor(tom, pat) ✓

Answer: YES — tom is an ancestor of pat, derived automatically through chained rules.

Another Example — Employee Hierarchy:

Facts:

```
manages(alice, bob).    -- alice manages bob
manages(bob, carol).    -- bob manages carol
manages(carol, dave).   -- carol manages dave
```

Rule:

```
senior(X, Y) :- manages(X, Y).
senior(X, Y) :- manages(X, Z), senior(Z, Y).
```

Query: `?- senior(alice, dave).` **Result:** YES (alice $\rightarrow$ bob $\rightarrow$ carol $\rightarrow$ dave)

Advantages of Deductive Databases:

- Can answer complex recursive queries easily (like finding all ancestors)
- More expressive than SQL for reasoning
- Can derive information not explicitly stored
- Good for expert systems and knowledge bases

Disadvantages:

- More complex to design and use
- Slower for simple queries compared to SQL
- Less popular in industry (used mainly in research)

Examples of Deductive DBMS:

- LDL++ (Logic Data Language)
 - Coral
 - XSB
 - Datalog systems
-

Q17. Describe 3-Schema Architecture in DBMS.

The Three-Schema Architecture is a framework that organizes a DBMS into three separate levels (schemas), so that changes at one level don't affect other levels. This is the foundation for data independence.

Proposed by ANSI/SPARC in 1975. The goal is to separate the user's view of data from the physical storage of data.

The Three Levels:

LEVEL 1: External Level (View Level / User Level)

This is the **topmost level** — what individual users or user groups see.

- Different users can see different views of the same database
- Each user only sees the data relevant to them
- Hides complexity from end users
- Also called **subschema** or **user schema**

Example:

- A student sees only their own marks and courses
- An admin sees all students' data
- An accountant sees only financial records
- All are looking at the same underlying database but through different views

Multiple external schemas can exist for one database.

LEVEL 2: Conceptual Level (Logical Level)

This is the **middle level** — the complete logical view of the entire database.

- Describes WHAT data is stored and the relationships between them
- Hides physical storage details
- One conceptual schema per database (unlike multiple external schemas)
- Created and managed by Database Designer/DBA

Includes:

- All entities (tables)
- All attributes (columns)
- All relationships between tables
- Integrity constraints and security rules
- Does NOT include how data is physically stored

Example:

```
STUDENT(SID, NAME, AGE, DEPT)
COURSE(CID, CNAME, CREDITS)
ENROLLED(SID, CID, GRADE)
```

LEVEL 3: Internal Level (Physical Level)

This is the **lowest level** — how data is actually stored on the physical disk.

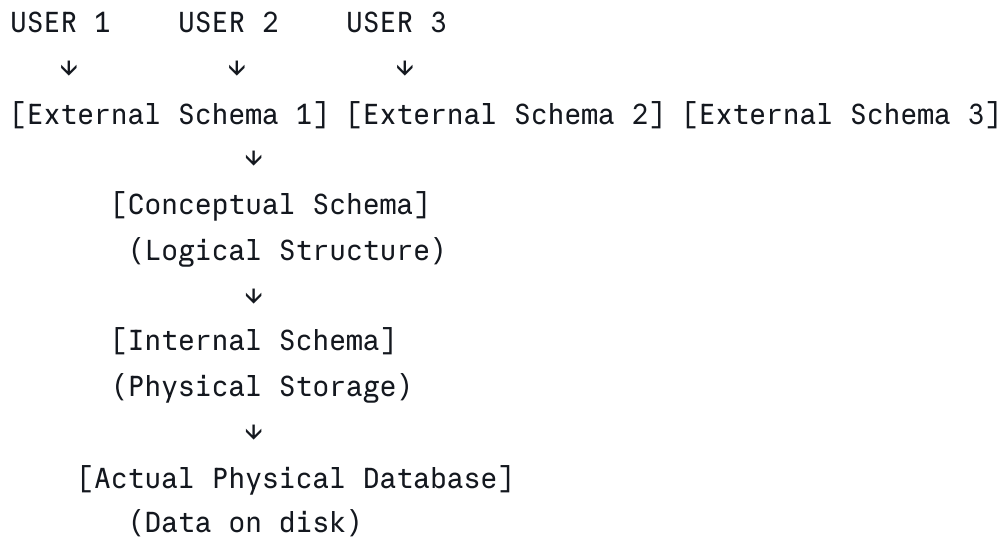
- Describes the physical storage structure of the database
- Includes file formats, indexing methods, storage allocation
- Deals with actual bytes on disk
- Managed by the DBMS software internally

Includes:

- Physical file organization (sequential, hash, B-tree)
- Index structures (which columns are indexed)
- Storage allocation and compression
- Record lengths and block sizes

Example: "Student data is stored in a B-tree indexed file, sorted by SID, with each record taking 200 bytes."

How the Three Levels Work Together:



Mappings Between Levels:

External-Conceptual Mapping:

- Connects each user's view to the conceptual schema
- When the conceptual schema changes, only this mapping changes — user views remain unaffected (Logical Data Independence)

Conceptual-Internal Mapping:

- Connects the conceptual schema to the physical storage
- When physical storage changes, only this mapping changes — conceptual schema unaffected (Physical Data Independence)

Data Independence (the main benefit):

Logical Data Independence:

- Ability to change the conceptual schema without changing external schemas
- Example: Adding a new column to a table doesn't break existing user views

Physical Data Independence:

- Ability to change the internal schema without changing the conceptual schema

- Example: Changing from one storage format to another doesn't affect the table structure

Summary Table:

Level	Also Called	What it Describes	Who Manages It
External	View/User Level	What each user sees	DBA / Application
Conceptual	Logical Level	Entire DB structure	DBA / Designer
Internal	Physical Level	How data is stored on disk	DBMS Software

Q18. Explain Database Languages in DBMS.

Database languages are specialized languages used to communicate with a DBMS — to define, manipulate, control, and query data.

SQL (Structured Query Language) is the most common, and it contains several sub-languages, each with a specific purpose.

1. DDL — Data Definition Language

Purpose: Used to **define and modify the structure (schema)** of the database — creating tables, modifying them, deleting them.

DDL commands deal with the **structure**, not the data itself.

Commands:

CREATE — Create a new database object (table, index, view)

```
sql
```

```
CREATE TABLE STUDENT (  
  SID INT PRIMARY KEY,  
  NAME VARCHAR(50) NOT NULL,  
  AGE INT,  
  DEPT VARCHAR(20)  
);
```

ALTER — Modify existing structure (add/remove/modify columns)

sql

```
ALTER TABLE STUDENT ADD EMAIL VARCHAR(100);  
ALTER TABLE STUDENT MODIFY AGE DECIMAL(3,0);
```

DROP — Permanently delete a table or other object

sql

```
DROP TABLE STUDENT;
```

TRUNCATE — Delete all data from a table but keep the structure

sql

```
TRUNCATE TABLE STUDENT;
```

RENAME — Rename a table

sql

```
RENAME TABLE STUDENT TO LEARNER;
```

2. DML — Data Manipulation Language

Purpose: Used to **insert, update, delete, and retrieve data** in the database tables.

DML works with the **data inside the tables**.

Commands:

INSERT — Add new rows to a table

sql

```
INSERT INTO STUDENT (SID, NAME, AGE, DEPT)  
VALUES (101, 'Raj', 20, 'CS');
```

SELECT — Retrieve data from one or more tables

sql

```
SELECT NAME, DEPT FROM STUDENT WHERE AGE > 18;
```

```
sql
```

```
UPDATE STUDENT SET DEPT = 'EE' WHERE SID = 101;
```

DELETE — Remove rows from a table

```
sql
```

```
DELETE FROM STUDENT WHERE SID = 101;
```

3. DCL — Data Control Language

Purpose: Used to **control access to data** — grant or revoke permissions to users.

Commands:

GRANT — Give a user permission to perform certain operations

```
sql
```

```
GRANT SELECT, INSERT ON STUDENT TO user123;
```

This gives user123 the ability to read and insert data into the STUDENT table.

REVOKE — Remove previously given permissions

```
sql
```

```
REVOKE INSERT ON STUDENT FROM user123;
```

This removes user123's ability to insert into the STUDENT table.

4. TCL — Transaction Control Language

Purpose: Used to **manage transactions** — control when changes are saved permanently or undone.

Commands:

COMMIT — Permanently save all changes made in the current transaction

```
sql
```

```
UPDATE ACCOUNT SET BALANCE = BALANCE - 5000 WHERE ACC_ID = 1;
UPDATE ACCOUNT SET BALANCE = BALANCE + 5000 WHERE ACC_ID = 2;
COMMIT; -- Now the transfer is permanent
```

ROLLBACK — Undo all changes made in the current transaction

```
sql

UPDATE ACCOUNT SET BALANCE = BALANCE - 5000 WHERE ACC_ID = 1;
-- Something went wrong!
ROLLBACK; -- Undo the deduction
```

SAVEPOINT — Set a point within a transaction to rollback to

```
sql

SAVEPOINT sp1;
DELETE FROM STUDENT WHERE DEPT = 'CS';
ROLLBACK TO sp1; -- Only undo the delete, not earlier operations
```

5. DQL — Data Query Language

Purpose: Specifically for **querying and retrieving data**. Some consider SELECT as a separate category.

SELECT with various clauses:

```
sql

-- WHERE: filter rows
SELECT * FROM EMPLOYEE WHERE SALARY > 50000;

-- ORDER BY: sort results
SELECT NAME, SALARY FROM EMPLOYEE ORDER BY SALARY DESC;

-- GROUP BY: group rows for aggregate functions
SELECT DEPT, COUNT(*) FROM EMPLOYEE GROUP BY DEPT;

-- HAVING: filter after grouping (like WHERE but for groups)
SELECT DEPT, AVG(SALARY) FROM EMPLOYEE
GROUP BY DEPT HAVING AVG(SALARY) > 60000;
```

```
-- JOIN: combine data from multiple tables
SELECT E.NAME, D.DNAME
FROM EMPLOYEE E
JOIN DEPARTMENT D ON E.DEPTID = D.ID;
```

Summary Table:

Language	Full Form	Purpose	Key Commands
DDL	Data Definition Language	Define/modify structure	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Work with data	INSERT, SELECT, UPDATE, DELETE
DCL	Data Control Language	Access control	GRANT, REVOKE
TCL	Transaction Control Language	Manage transactions	COMMIT, ROLLBACK, SAVEPOINT
DQL	Data Query Language	Query data	SELECT

Q19. Describe Aggregate Functions with an Example.

Aggregate functions perform calculations on a set of rows and return a single summarized value. They are used to get summary information from data.

Instead of showing each individual row, aggregate functions give you totals, averages, counts, etc.

The 5 Main Aggregate Functions:

Sample Table: PRODUCT_MAST

PROD_ID	PRODUCT_NAME	COMPANY	RATE	QUANTITY
1	Pen	Natraj	10	100
2	Pencil	Natraj	5	200
3	Notebook	Classmate	40	50

PROD_ID	PRODUCT_NAME	COMPANY	RATE	QUANTITY
4	Eraser	Natraj	8	150
5	Ruler	Classmate	25	80

1. COUNT() — Count the number of rows

Counts how many rows exist (or how many non-null values in a column).

```
sql

-- Count total number of products
SELECT COUNT(*) FROM PRODUCT_MAST;
-- Result: 5

-- Count with condition
SELECT COUNT(*) FROM PRODUCT_MAST WHERE RATE >= 10;
-- Result: 3 (Pen=10, Notebook=40, Ruler=25)

-- Count distinct companies
SELECT COUNT(DISTINCT COMPANY) FROM PRODUCT_MAST;
-- Result: 2 (Natraj, Classmate)

-- Count products per company
SELECT COMPANY, COUNT(*) FROM PRODUCT_MAST GROUP BY COMPANY;
-- Result: Natraj=3, Classmate=2
```

2. SUM() — Add up all values

Calculates the total sum of a numeric column.

```
sql

-- Total value of all products
SELECT SUM(RATE * QUANTITY) FROM PRODUCT_MAST;
-- Result: 10*100 + 5*200 + 40*50 + 8*150 + 25*80 = 1000+1000+2000+1200+2000 = 7200

-- Total rate of all products
SELECT SUM(RATE) FROM PRODUCT_MAST;
-- Result: 10+5+40+8+25 = 88

-- Sum per company
```

```
SELECT COMPANY, SUM(RATE) FROM PRODUCT_MAST GROUP BY COMPANY;  
-- Result: Natraj=23, Classmate=65
```

3. AVG() — Calculate the average

Calculates the arithmetic mean (average) of a numeric column. Ignores NULL values.

```
sql  
  
-- Average rate of all products  
SELECT AVG(RATE) FROM PRODUCT_MAST;  
-- Result: 88/5 = 17.6  
  
-- Average per company  
SELECT COMPANY, AVG(RATE) FROM PRODUCT_MAST GROUP BY COMPANY;  
-- Result: Natraj = (10+5+8)/3 = 7.67, Classmate = (40+25)/2 = 32.5
```

4. MAX() — Find the maximum value

Returns the largest value in a column.

```
sql  
  
-- Most expensive product rate  
SELECT MAX(RATE) FROM PRODUCT_MAST;  
-- Result: 40 (Notebook)  
  
-- Most expensive product name  
SELECT PRODUCT_NAME, RATE FROM PRODUCT_MAST  
WHERE RATE = (SELECT MAX(RATE) FROM PRODUCT_MAST);  
-- Result: Notebook, 40
```

5. MIN() — Find the minimum value

Returns the smallest value in a column.

```
sql
```

```
-- Cheapest product rate
SELECT MIN(RATE) FROM PRODUCT_MAST;
-- Result: 5 (Pencil)

-- Cheapest product per company
SELECT COMPANY, MIN(RATE) FROM PRODUCT_MAST GROUP BY COMPANY;
-- Result: Natraj=5, Classmate=25
```

Using GROUP BY with Aggregate Functions:

GROUP BY divides rows into groups and the aggregate function is applied to each group separately.

```
sql

SELECT COMPANY, COUNT(*), SUM(RATE), AVG(RATE), MAX(RATE), MIN(RATE)
FROM PRODUCT_MAST
GROUP BY COMPANY;
```

COMPANY	COUNT	SUM	AVG	MAX	MIN
Natraj	3	23	7.67	10	5
Classmate	2	65	32.5	40	25

Using HAVING to Filter Groups:

HAVING is like WHERE but works after GROUP BY — it filters groups, not individual rows.

```
sql

-- Show companies that have more than 2 products
SELECT COMPANY, COUNT(*) FROM PRODUCT_MAST
GROUP BY COMPANY HAVING COUNT(*) > 2;
-- Result: Only Natraj (has 3 products)
```

Difference:

- WHERE filters individual rows BEFORE grouping
- HAVING filters groups AFTER grouping

Q20. SQL Queries for EMPLOYEE-DEPARTMENT-PROJECT Schema.

Given Schema:

- EMPLOYEE (EID, ENAME, ADDRESS, SALARY, DEPTID)
 - DEPARTMENT (ID, DNAME, HOD)
 - PROJECT (PID, PNAME, DEPTID, LOCATION)
 - DEPENDENT (NAME, EID, AGE)
 - WORK_ON (PID, EID, HOURS)
-

i. Retrieve the name and address of employees who work for department no 5.

sql

```
SELECT ENAME, ADDRESS  
FROM EMPLOYEE  
WHERE DEPTID = 5;
```

This simply filters employees where DEPTID equals 5 and shows their name and address.

ii. Retrieve the id of employee who has NO dependents.

sql

```
SELECT EID  
FROM EMPLOYEE  
WHERE EID NOT IN (  
    SELECT EID FROM DEPENDENT  
);
```

The inner query finds all EIDs that appear in the DEPENDENT table (employees who have dependents). The outer query finds employees whose EID is NOT in that list — i.e., employees with no dependents.

iii. Retrieve name and department id of employees from 'Bengaluru' with salary greater than 60000.

sql

```
SELECT ENAME, DEPTID
FROM EMPLOYEE
WHERE ADDRESS = 'Bengaluru'
AND SALARY > 60000;
```

Both conditions must be true: employee must be from Bengaluru AND salary must exceed 60000.

iv. Find all names of employees who work on ALL projects controlled by department 3.

sql

```
SELECT ENAME
FROM EMPLOYEE E
WHERE NOT EXISTS (
    SELECT PID FROM PROJECT
    WHERE DEPTID = 3
    AND PID NOT IN (
        SELECT PID FROM WORK_ON
        WHERE EID = E.EID
    )
);
```

How this works:

- The inner-most query: Find projects of dept 3 that this employee is NOT working on
 - NOT EXISTS: Select the employee only if there are NO such missing projects
 - Result: Employees who work on every single project of department 3
-

v. Find employee names who work under HOD "Dr. ABC".

sql

```
SELECT E.ENAME
FROM EMPLOYEE E
JOIN DEPARTMENT D ON E.DEPTID = D.ID
WHERE D.HOD = 'Dr. ABC';
```

Q21. Explain Advantages and Disadvantages of Views in DBMS.

A View is a virtual table created by a SELECT query. It does not store data itself — it just shows data from the underlying base tables.

Think of a view like a window — you see through it to the actual data, but the window itself doesn't hold anything.

Creating a view:

sql

```
CREATE VIEW CS_Students AS
SELECT SID, NAME, AGE
FROM STUDENT
WHERE DEPT = 'CS';
```

Advantages of Views:

1. Data Security Views hide sensitive columns from users who don't need to see them.

Example: The HR view shows employee names and departments but hides salary information. Different departments see only what they need.

sql

```
CREATE VIEW Employee_Public AS
SELECT ENAME, DEPT FROM EMPLOYEE;
-- Salary column is hidden
```

2. Simplifies Complex Queries A complex query with multiple joins can be saved as a view. Users can then query the view simply without knowing the complexity behind it.

Example: Instead of writing a 5-table join every time, create a view:

sql

```
CREATE VIEW Employee_Info AS
SELECT E.ENAME, D.DNAME, P.PNAME
FROM EMPLOYEE E
```

```
JOIN DEPARTMENT D ON E.DEPTID = D.ID
JOIN WORK_ON W ON E.EID = W.EID
JOIN PROJECT P ON W.PID = P.PID;

-- Now users just write:
SELECT * FROM Employee_Info;
```

3. Logical Data Independence If the underlying table structure changes (like a column renamed), you can update the view definition and user applications continue to work without changes.

4. Consistent View of Data Multiple users can have their own customized view of the same data without affecting others.

5. Reduces Storage Views don't store data — they only store the query definition. The actual data stays in the base tables. So no extra storage is needed.

6. Reusability A view defined once can be used many times by many different queries and users.

7. Data Integrity When users insert/update data through a view, the DBMS can automatically check integrity constraints.

Disadvantages of Views:

1. Performance Overhead Every time a view is queried, the underlying SELECT query is re-executed. For complex views with many joins, this can be slow.

2. Update Restrictions You cannot always INSERT, UPDATE, or DELETE data through a view. Restrictions include:

- Cannot update if the view uses GROUP BY or DISTINCT
- Cannot insert if the base table has NOT NULL columns not in the view
- Cannot update if the view involves multiple base tables
- Cannot use views created with read-only option

3. Dependency on Base Tables If the base table is dropped or altered, the view may become invalid. Views depend on the structure of their underlying tables.

4. Cannot Pass Parameters SQL views cannot accept parameters — they always show the same data (unless you use WHERE conditions in the view definition).

5. Cannot Use Temporary Tables Views cannot be created on temporary tables.

7. Hidden Complexity For beginners, it may be confusing that a "table" they're querying is actually a view with hidden complexity.

Types of Views:

1. Simple View — Based on one table, no group functions, allows DML operations
2. Complex View (Join View) — Based on multiple tables, may restrict some DML
3. Inline View — A subquery used in the FROM clause of another query

Summary:

Views	
✓ Good for	Security, simplicity, consistency
✗ Bad for	Heavy updates, performance-critical operations

Q22. Create Triggers for LOAN_ACCOUNT and PAYMENT_LOG.

(This is a repeat of Q8 — same question, same answer)

Given tables:

- LOAN_ACCOUNT (LOAN_ID, CUST_ID, LOAN_AMOUNT, PAID_AMOUNT, STATUS)
 - PAYMENT_LOG (LOG_ID, LOAN_ID, OLD_PAID, NEW_PAID, PAYMENT_DATE)
-

iii. Trigger to PREVENT payment if PAID_AMOUNT > LOAN_AMOUNT:

```
sql

CREATE TRIGGER prevent_overpayment
BEFORE UPDATE ON LOAN_ACCOUNT
FOR EACH ROW
BEGIN
    IF NEW.PAID_AMOUNT > NEW.LOAN_AMOUNT THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Paid amount cannot exceed loan amount!';
```

```
END IF;  
END;
```

Explanation:

- Fires BEFORE any UPDATE on LOAN_ACCOUNT
- NEW.PAID_AMOUNT = the value being set
- If this new value exceeds LOAN_AMOUNT, an error is raised and update is blocked
- The bank can never accidentally record payment beyond loan amount

iv. Trigger to RECORD payment in PAYMENT_LOG:

```
sql  
  
CREATE TRIGGER log_payment  
AFTER UPDATE ON LOAN_ACCOUNT  
FOR EACH ROW  
BEGIN  
    IF OLD.PAID_AMOUNT <> NEW.PAID_AMOUNT THEN  
        INSERT INTO PAYMENT_LOG  
            (LOAN_ID, OLD_PAID, NEW_PAID, PAYMENT_DATE)  
        VALUES  
            (OLD.LOAN_ID, OLD.PAID_AMOUNT, NEW.PAID_AMOUNT, NOW());  
    END IF;  
END;
```

Explanation:

- Fires AFTER UPDATE (only after successful update)
- OLD.PAID_AMOUNT = value before update
- NEW.PAID_AMOUNT = value after update
- Only logs if PAID_AMOUNT actually changed
- NOW() records the current date and time of payment

Q23. Describe Different Types of Functional Dependencies with an Example.

(Repeat of Q9 — same answer)

Please refer to **Q9 answer above** — it covers all 6 types: Full FD, Partial FD, Transitive FD, Trivial FD, Non-Trivial FD, and Multi-Valued FD with complete examples.

Q24. Compute Closure for $R=\{A,B,C,D,E\}$, FDs: $\{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$. Find Candidate Keys and Superkeys.

Given FDs:

- $A \rightarrow BC$
 - $CD \rightarrow E$
 - $B \rightarrow D$
 - $E \rightarrow A$
-

Step 1: Find Closure of Single Attributes

A^+ :

- Start: $\{A\}$
- $A \rightarrow BC \rightarrow$ add B, C $\rightarrow \{A, B, C\}$
- $B \rightarrow D \rightarrow$ add D $\rightarrow \{A, B, C, D\}$
- $CD \rightarrow E$ (have C and D) $\rightarrow$ add E $\rightarrow \{A, B, C, D, E\}$
- $A^+ = \{A, B, C, D, E\}$  A alone covers all attributes!

Since A alone is a superkey, and no proper subset of A (just empty set) is a superkey, **A is a Candidate Key!**

B^+ :

- Start: $\{B\}$
 - $B \rightarrow D \rightarrow$ add D $\rightarrow \{B, D\}$
 - No more FDs fire
 - $B^+ = \{B, D\}$ — Not a superkey
-

C^+ :

- Start: $\{C\}$
- No FD has C alone on the left side

- $C^+ = \{C\}$ — Not a superkey
-

D⁺:

- Start: $\{D\}$
 - No FD has D alone on left
 - $D^+ = \{D\}$ — Not a superkey
-

E⁺:

- Start: $\{E\}$
- $E \rightarrow A \rightarrow \text{add } A \rightarrow \{E, A\}$
- $A \rightarrow BC \rightarrow \text{add } B, C \rightarrow \{A, B, C, E\}$
- $B \rightarrow D \rightarrow \text{add } D \rightarrow \{A, B, C, D, E\}$
- $E^+ = \{A, B, C, D, E\}$  E alone covers all!

No proper subset of E covers all $\rightarrow$ **E is a Candidate Key!**

Check other possible candidate keys (combinations):

CD⁺:

- Start: $\{C, D\}$
- $CD \rightarrow E \rightarrow \text{add } E \rightarrow \{C, D, E\}$
- $E \rightarrow A \rightarrow \text{add } A \rightarrow \{A, C, D, E\}$
- $A \rightarrow BC \rightarrow \text{add } B \rightarrow \{A, B, C, D, E\}$
- $CD^+ = \{A, B, C, D, E\}$ 
- Is C alone or D alone a superkey? No ($C^+ = \{C\}$, $D^+ = \{D\}$)
- So CD is minimal $\rightarrow$ **CD is a Candidate Key!**

BC⁺:

- Start: $\{B, C\}$
- $B \rightarrow D \rightarrow \text{add } D \rightarrow \{B, C, D\}$
- $CD \rightarrow E \rightarrow \text{add } E \rightarrow \{B, C, D, E\}$
- $E \rightarrow A \rightarrow \text{add } A \rightarrow \{A, B, C, D, E\}$

- $BC^+ = \{A, B, C, D, E\}$ ✓
 - B alone? $B^+ = \{B, D\}$ — No. C alone? $C^+ = \{C\}$ — No.
 - **BC is a Candidate Key!**
-

i) Candidate Keys for R:

- A
 - E
 - CD
 - BC
-

ii) Superkeys:

(Every superset of a candidate key is a superkey)

From candidate key A: A, AB, AC, AD, AE, ABC, ABD, ABE, ACD, ACE, ADE, ABCD, ABCE, ABDE, ACDE, ABCDE

From candidate key E: E, BE, CE, DE, BCE, BDE, CDE, BCDE (and with A already covered above)

From CD: CD, ACD, BCD, CDE, ABCD, BCDE, ACDE, ABCDE

Some key superkeys: A, E, BC, CD, AB, AC, AE, BE, CE, DE, ABC, ABD, ACD, BCD, CDE, BCE... (all combinations that include at least one candidate key)

Q25. Explain in Detail Transaction States with a Neat Diagram.

A transaction goes through several states from the time it starts until it finishes. Understanding these states is important for database recovery and concurrency control.

A transaction is a sequence of operations (read/write) that must be completed as a unit.

The 5 Transaction States:

1. Active State

- The transaction **starts and is currently executing**

- Read and write operations are being performed
- This is the initial state of every transaction
- Most of the transaction's lifetime is spent here

Example: A bank transfer starts — the system is reading the balance and preparing to deduct.

2. Partially Committed State

- The transaction has **executed its last operation** but changes are not yet permanently saved
- The changes exist in memory (buffer), not yet written to disk
- The system checks if everything is okay before committing

Example: The deduction from Account A is done in memory, waiting to be written to disk.

3. Committed State

- The transaction has **successfully completed** and all changes are permanently saved to the database
- Changes are written to disk
- Transaction cannot be rolled back after this point
- This is a successful completion

Example: Transfer is complete. Account A shows ₹5000 less and Account B shows ₹5000 more — permanently saved.

4. Failed State

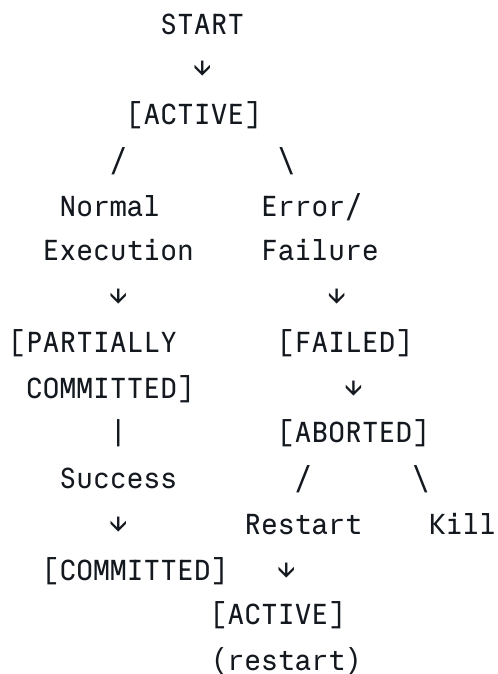
- The transaction **cannot continue** due to an error
- Could be hardware failure, software error, or constraint violation
- The transaction must be aborted and rolled back

Example: While transferring money, power goes out or a network error occurs.

5. Aborted State

Example: The partial deduction from Account A is undone. Balance returns to original value.

Transaction State Diagram:



State Transitions Explained:

From State	To State	When
Active	Partially Committed	Last operation executed successfully
Active	Failed	Error occurs during execution
Partially Committed	Committed	Changes written to disk successfully
Partially Committed	Failed	Error occurs during final write

From State	To State	When
Failed	Aborted	Rollback is completed
Aborted	Active	Transaction restarted

Rollback and Recovery:

When a transaction is aborted, the DBMS uses the **Undo Log** to reverse all changes:

- Uses write-ahead logging (WAL) — logs are written before actual data changes
- The log records old values and new values of each changed item
- During rollback, old values are restored

Q26. Illustrate Problems that Occur During Concurrent Transactions.

When multiple transactions run at the same time (concurrently), several problems can occur if they are not properly controlled.

Concurrency is needed for performance (multiple users accessing DB at same time), but without control, it causes data problems.

Problem 1: Lost Update Problem (Write-Write Conflict)

Two transactions read the same value and both update it. The second update overwrites the first one — the first update is "lost."

Example:

Time	Transaction T1	Transaction T2	Balance
t1	Read Balance = 1000		1000
t2		Read Balance = 1000	1000
t3	Balance = $1000 - 200 = 800$		800
t4	Write Balance = 800		800
t5		Balance = $1000 - 100 = 900$	
t6		Write Balance = 900	900

Problem: T1 withdrew ₹200 but the final balance is 900 instead of 700. T1's update is lost!

Problem 2: Dirty Read Problem (Read-Write Conflict)

Transaction T2 reads data that was modified by T1, but T1 later rolls back. T2 has read "dirty" (invalid) data.

Example:

Time	T1	T2	Balance
t1	Write Balance = 800		800
t2		Read Balance = 800	800
t3	ROLLBACK → Balance = 1000		1000
t4		Use 800 for calculation	Wrong!

Problem: T2 used balance 800 which is invalid — the actual balance is 1000 after T1 rolled back.

Problem 3: Unrepeatable Read (Inconsistent Retrievals)

Transaction T1 reads the same data twice and gets different values because T2 modified the data between the two reads.

Example:

Time	T1	T2	Balance
t1	Read Balance = 1000		1000
t2		Write Balance = 800	800
t3		COMMIT	800
t4	Read Balance = 800		800

Problem: T1 reads balance as 1000 first time and 800 second time in the same transaction — inconsistent!

Problem 4: Phantom Read

T1 executes a query and gets a set of rows. T2 then inserts new rows matching T1's query condition. If T1 runs the same query again, it gets more rows — the new rows are "phantom" (unexpected) rows.

Example:

- T1 queries: `SELECT * FROM ORDERS WHERE AMOUNT > 5000` → Gets 3 orders
- T2 inserts a new order with `AMOUNT = 6000` and commits
- T1 queries again: Same query → Gets 4 orders now!

Problem: T1 sees different numbers of rows in the same transaction.

Problem 5: Deadlock

Two transactions each hold a lock and wait for the other to release its lock. Neither can proceed — they are stuck forever.

Example:

Time	T1	T2
t1	Lock(Account_A)	
t2		Lock(Account_B)
t3	Wait for Lock(Account_B)	
t4		Wait for Lock(Account_A)

Problem: T1 waits for T2 and T2 waits for T1 — both are stuck in a deadlock!

Solution: Deadlock detection (kill one transaction) or deadlock prevention (ordering resource acquisition).

Solutions to Concurrency Problems:

Problem	Solution
Lost Update	Locking mechanisms (exclusive locks)

Problem	Solution
Dirty Read	Strict 2-Phase Locking, isolation levels
Unrepeatable Read	Snapshot isolation, repeatable read isolation
Phantom Read	Predicate locking, serializable isolation
Deadlock	Deadlock detection + recovery, timeout mechanisms

Q27. Model an ER Diagram for the COMPANY Database.

Scenario:

- Company organized into departments
- Each department has employees, a manager, and supervisors
- Supervisors supervise a set of employees
- Departments control projects
- Employees work on projects (multiple projects per day)
- Company tracks dependents of each employee

Entities and Attributes:

EMPLOYEE: EMP_ID (key), ENAME, ADDRESS, SALARY, DOB, GENDER

DEPARTMENT: DEPT_ID (key), DNAME, DLOCATION **PROJECT:** PROJ_ID (key),

PNAME, PLOCATION **DEPENDENT:** DEP_NAME (partial key), RELATIONSHIP, DOB, GENDER

Relationships:

Relationship	Between	Cardinality	Notes
WORKS_FOR	EMPLOYEE — DEPARTMENT	N:1	Many employees, one dept
MANAGES	EMPLOYEE — DEPARTMENT	1:1	One manager per dept
SUPERVISES	EMPLOYEE — EMPLOYEE	1:N	Recursive — supervisor & subordinate

Relationship	Between	Cardinality	Notes
WORKS_ON	EMPLOYEE — PROJECT	M:N	Many employees, many projects
CONTROLS	DEPARTMENT — PROJECT	1:N	One dept controls many projects
HAS_DEPENDENT	EMPLOYEE — DEPENDENT	1:N	Weak relationship

Special Notes for ER Diagram:

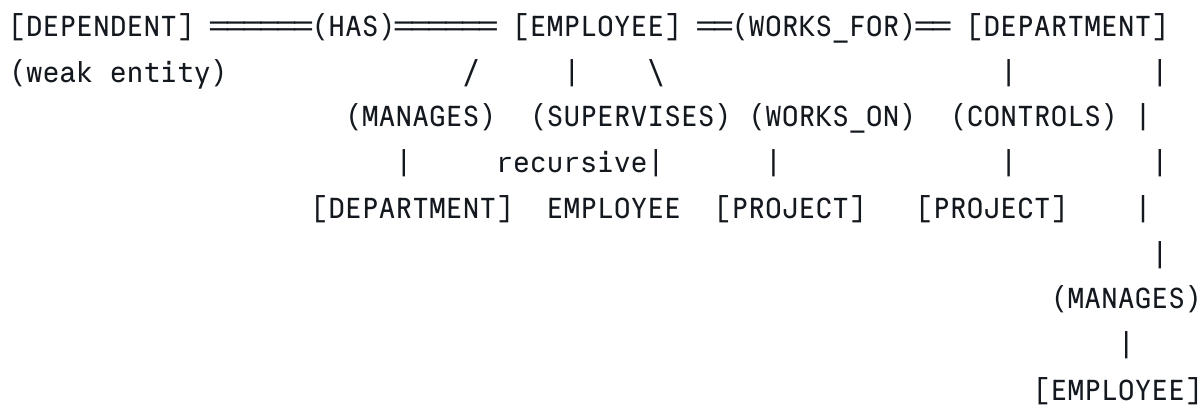
SUPERVISES is a **recursive relationship** (also called unary relationship) — the EMPLOYEE entity participates twice, once as SUPERVISOR and once as SUPERVISEE. Both roles must be labeled.

DEPENDENT is a **Weak Entity** — shown with double rectangle. It depends on EMPLOYEE to exist. Its partial key is DEP_NAME.

HAS_DEPENDENT is an **Identifying Relationship** — shown with double diamond.

WORKS_ON has the attribute **HOURS** (hours worked on that project each day) and **DATE** — these are relationship attributes.

ER Diagram Description:



Participation Constraints:

- EMPLOYEE → WORKS_FOR → DEPARTMENT: Total (every employee must work in a dept)
- DEPARTMENT → MANAGES → EMPLOYEE: Partial (not every employee is a manager)
- DEPENDENT: Total participation in HAS (dependents must belong to an employee)

Q28. Set Operations on SAVINGS_ACCOUNT and CURRENT_ACCOUNT.

Given:

SAVINGS_ACCOUNT:

CUST_ID	NAME
101	Ravi
102	Anjali
103	Kiran
104	Meena
105	Arjun

CURRENT_ACCOUNT:

CUST_ID	NAME
103	Kiran
104	Meena
106	Suresh
107	Neha
108	Rahul

i) SAVINGS_ACCOUNT $\cup$ CURRENT_ACCOUNT (UNION — All unique records from both)

CUST_ID	NAME
101	Ravi
102	Anjali
103	Kiran

CUST_ID	NAME
104	Meena
105	Arjun
106	Suresh
107	Neha
108	Rahul

Total 8 unique records. Kiran (103) and Meena (104) appear only once.

ii) SAVINGS_ACCOUNT $\cap$ CURRENT_ACCOUNT (INTERSECTION — Common in BOTH)

CUST_ID	NAME
103	Kiran
104	Meena

Only Kiran and Meena have both savings and current accounts.

iii) SAVINGS_ACCOUNT $\bowtie$ CURRENT_ACCOUNT (NATURAL JOIN — Match on common columns)

Common columns: CUST_ID and NAME. Natural join returns rows where both CUST_ID and NAME match.

CUST_ID	NAME
103	Kiran
104	Meena

Same as intersection here since the join attributes are the only columns in both tables.

iv) SAVINGS_ACCOUNT – CURRENT_ACCOUNT (SET DIFFERENCE — In Savings but NOT in Current)

CUST_ID	NAME
101	Ravi
102	Anjali
105	Arjun

These 3 customers have savings accounts but no current accounts.

v) SAVINGS_ACCOUNT × CURRENT_ACCOUNT (CARTESIAN PRODUCT)

5 rows × 5 rows = **25 rows** total.

SA.CUST_ID	SA.NAME	CA.CUST_ID	CA.NAME
101	Ravi	103	Kiran
101	Ravi	104	Meena
101	Ravi	106	Suresh
101	Ravi	107	Neha
101	Ravi	108	Rahul
102	Anjali	103	Kiran
102	Anjali	104	Meena
102	Anjali	106	Suresh
102	Anjali	107	Neha
102	Anjali	108	Rahul
103	Kiran	103	Kiran
...	...	...	...

(continues for all 25 combinations)

Q29. Explain Data Models in DBMS.

A Data Model is a collection of concepts and rules used to describe the structure of a database — what data is stored, how it is organized, and what operations can be performed on it.

It acts as a blueprint showing how data is organized and related.

Categories of Data Models:

1. Conceptual (High-Level) Data Models

Also called entity-based or semantic data models. Close to how users think about data. Independent of how data will be stored in the computer.

Entity-Relationship (ER) Model:

- Most popular conceptual model
- Uses Entities (real-world objects), Attributes (properties), and Relationships (connections)
- Represented visually using ER diagrams
- Example: STUDENT entity with attributes (SID, NAME, AGE) connected to COURSE via ENROLLS relationship

Object-Based Data Model:

- Represents data as objects (similar to OOP)
 - Supports encapsulation, inheritance, polymorphism
 - Used in OODBMS
-

2. Implementation (Representational) Data Models

These are in between conceptual and physical. Used in actual DBMS implementations.

Relational Model:

- Most widely used model (MySQL, Oracle, PostgreSQL)
- Data organized in **tables (relations)** with rows and columns
- Tables connected through foreign keys

- Based on mathematical concept of sets

Example:

STUDENT Table:

SID	NAME	AGE	DEPT
101	Raj	20	CS
102	Sneha	21	EE

Network Model:

- Data organized as a graph — records are nodes, relationships are edges
- A child record can have multiple parent records (M:N relationships)
- Uses pointers to link records
- Example: IDS/II, IDMS

Hierarchical Model:

- Data organized as a tree structure (parent-child)
- Each child has exactly one parent (but a parent can have many children)
- Like a family tree or company org chart
- Example: IBM's IMS

3. Physical (Low-Level) Data Models

Describe how data is actually stored on disk — file formats, access paths, block sizes. Used internally by the DBMS.

Comparison of Main Data Models:

Feature	Hierarchical	Network	Relational	Object-Oriented
Structure	Tree	Graph	Tables	Objects
Relationships	1:N only	M:N supported	Foreign keys	References
Language	Proprietary	Proprietary	SQL	OQL
Complexity	Simple	Complex	Medium	Complex

Feature	Hierarchical	Network	Relational	Object-Oriented
Example DBMS	IMS	IDMS	MySQL, Oracle	db4o, ObjectDB
Best for	Simple hierarchical data	Complex networks	Most business applications	Complex, multimedia data

Other Data Models:

Semi-Structured Data Model:

- Data that doesn't fit neatly into tables
- Uses XML or JSON format
- Used in web data, document stores (MongoDB)

Flat Data Model:

- Simplest model — data in a single table (like a spreadsheet)
- No relationships between tables
- Good only for very simple, small datasets

Q30. Describe 5 Constraints in SQL with Examples.

Constraints are rules applied to table columns to enforce data integrity — they prevent incorrect or inconsistent data from being stored.

1. NOT NULL Constraint

Ensures a column **cannot have empty (NULL) values**. Every record must have a value for this column.

```
sql

CREATE TABLE STUDENT (
  SID INT NOT NULL,
  NAME VARCHAR(50) NOT NULL,
  AGE INT
  -- AGE can be NULL (optional), but SID and NAME must always have values
);
```

If you try:

```
sql

INSERT INTO STUDENT VALUES (NULL, 'Raj', 20);
-- ERROR: SID cannot be NULL
```

Use case: Every student must have an ID and name — these cannot be left empty.

2. PRIMARY KEY Constraint

Uniquely identifies each row in a table. It is a combination of **NOT NULL** + **UNIQUE**. Only one primary key per table.

```
sql

CREATE TABLE EMPLOYEE (
  EMP_ID INT NOT NULL,
  ENAME VARCHAR(100),
  SALARY DECIMAL(10,2),
  PRIMARY KEY (EMP_ID)
);
```

- EMP_ID must be unique for every employee
- EMP_ID cannot be NULL
- No two employees can have the same EMP_ID

Use case: Each employee must have a unique identifier.

3. FOREIGN KEY Constraint

Ensures **referential integrity** — a column in one table must match values in a primary key column of another table. Links two tables together.

```
sql

CREATE TABLE ORDERS (
  ORDER_ID INT PRIMARY KEY,
  ORDER_DATE DATE,
  CUST_ID INT,
```

```
FOREIGN KEY (CUST_ID) REFERENCES CUSTOMER(CUST_ID)
);
```

- CUST_ID in ORDERS must exist in CUSTOMER table's CUST_ID column
- You cannot add an order for a customer that doesn't exist
- You cannot delete a customer if they have existing orders (prevents broken links)

Use case: An order must belong to an existing customer.

4. UNIQUE Constraint

Ensures all values in a column are **different from each other**. Unlike PRIMARY KEY, a table can have many UNIQUE constraints, and UNIQUE columns CAN have NULL (but only one NULL in most databases).

```
sql

CREATE TABLE STUDENT (
  SID INT PRIMARY KEY,
  NAME VARCHAR(50),
  EMAIL VARCHAR(100) UNIQUE,
  PHONE VARCHAR(15) UNIQUE
  -- Each student must have a unique email and unique phone number
);
```

- No two students can have the same email
- No two students can have the same phone number
- But SID is the primary key — EMAIL and PHONE are just UNIQUE

Use case: Two students cannot register with the same email.

5. CHECK Constraint

Ensures values in a column satisfy a **specific condition**. Custom validation rule.

```
sql

CREATE TABLE EMPLOYEE (
  EMP_ID INT PRIMARY KEY,
  ENAME VARCHAR(100),
```



```
AGE INT CHECK (AGE >= 18 AND AGE <= 65),
SALARY DECIMAL(10,2) CHECK (SALARY > 0),
GENDER CHAR(1) CHECK (GENDER IN ('M', 'F', 'O'))
);
```

- AGE must be between 18 and 65 (cannot hire a child or very old person)
- SALARY must be positive (cannot have negative salary)
- GENDER must be only 'M', 'F', or 'O'

If you try:

```
sql

INSERT INTO EMPLOYEE VALUES (101, 'Raj', 15, 30000, 'M');
-- ERROR: AGE fails the CHECK constraint (15 < 18)
```

Bonus — DEFAULT Constraint: Sets a default value when no value is provided.

```
sql

CREATE TABLE ORDERS (
  ORDER_ID INT PRIMARY KEY,
  STATUS VARCHAR(20) DEFAULT 'Pending',
  ORDER_DATE DATE DEFAULT GETDATE()
);

INSERT INTO ORDERS (ORDER_ID) VALUES (1001);
-- STATUS will automatically be 'Pending', ORDER_DATE will be today
```

Q31. SQL Queries for ACCOUNT Table Using Aggregate Functions.

Table: ACCOUNT(CUST_ID, NAME, BALANCE, TYPE, BRANCH)

i. Find the total balance held in the bank.

```
sql

SELECT SUM(BALANCE) AS Total_Bank_Balance
FROM ACCOUNT;
```

This adds up ALL balances from all customers across all branches.

ii. Find branches where total balance exceeds 10,00,000.

```
sql

SELECT BRANCH, SUM(BALANCE) AS Total_Balance
FROM ACCOUNT
GROUP BY BRANCH
HAVING SUM(BALANCE) > 1000000;
```

How it works:

- GROUP BY BRANCH: Groups all customers by their branch
 - SUM(BALANCE): Adds up balances for each branch
 - HAVING: Filters only those branches where total exceeds 10 lakhs
-

iii. Compare savings vs current account balances.

```
sql

SELECT TYPE,
        COUNT(*) AS Number_of_Accounts,
        SUM(BALANCE) AS Total_Balance,
        AVG(BALANCE) AS Average_Balance,
        MAX(BALANCE) AS Highest_Balance,
        MIN(BALANCE) AS Lowest_Balance
FROM ACCOUNT
GROUP BY TYPE;
```

This shows a complete comparison between SAVINGS and CURRENT account types.

iv. Find top branches based on average customer balance.

```
sql

SELECT BRANCH, AVG(BALANCE) AS Avg_Balance
FROM ACCOUNT
```

```
GROUP BY BRANCH
ORDER BY Avg_Balance DESC;
```

How it works:

- Calculates average balance per branch
- ORDER BY DESC: Shows highest average balance branch first
- The topmost result is the best-performing branch

v. Identify high-value customers per branch.

```
sql

SELECT BRANCH, NAME, BALANCE
FROM ACCOUNT A1
WHERE BALANCE = (
    SELECT MAX(BALANCE)
    FROM ACCOUNT A2
    WHERE A2.BRANCH = A1.BRANCH
)
ORDER BY BRANCH;
```

How it works:

- For each branch, find the customer with the maximum balance
- The subquery finds the max balance for the same branch
- Shows the high-value (richest) customer for each branch

Q32. Explain SQL Arithmetic and Logical Operators.

A. SQL Arithmetic Operators

Arithmetic operators perform **mathematical calculations** on numeric data.

Operator	Symbol	Description
Addition	+	Adds two values
Subtraction	-	Subtracts one value from another

Operator	Symbol	Description
Multiplication	*	Multiplies two values
Division	/	Divides one value by another
Modulus	%	Returns remainder after division

Examples using EMPLOYEE_DETAILS table (Emp_Salary column):

Addition (+): Add ₹20,000 bonus to existing salary

sql

```
SELECT Emp_Name, Emp_Salary + 20000 AS New_Salary
FROM Employee_Details;
```

Subtraction (-): Deduct ₹5,000 tax from salary

sql

```
SELECT Emp_Name, Emp_Salary - 5000 AS After_Tax_Salary
FROM Employee_Details;
```

Multiplication (*): Calculate annual salary (monthly × 12)

sql

```
SELECT Emp_Name, Emp_Salary * 12 AS Annual_Salary
FROM Employee_Details;
```

Division (/): Calculate half salary for part-time workers

sql

```
SELECT Emp_Name, Emp_Salary / 2 AS Part_Time_Salary
FROM Employee_Details;
```

Modulus (%): Check if employee number is odd or even

sql

```
SELECT Emp_ID, Emp_ID % 2 AS Remainder
FROM Employee_Details;
```

```
-- Remainder=0 means even, Remainder=1 means odd
```

B. SQL Logical Operators

Logical operators are used in **WHERE clauses** to combine or modify conditions.

1. AND Operator Returns rows where **ALL conditions are true**.

```
sql

-- Employees in Delhi with salary = 25000
SELECT * FROM Employee_Details
WHERE Emp_City = 'Delhi' AND Emp_Salary = 25000;
```

2. OR Operator Returns rows where **at least ONE condition is true**.

```
sql

-- Employees in Delhi OR with salary = 25000
SELECT * FROM Employee_Details
WHERE Emp_City = 'Delhi' OR Emp_Salary = 25000;
```

3. NOT Operator Reverses the condition — returns rows where condition is FALSE.

```
sql

-- Employees NOT from Delhi
SELECT * FROM Employee_Details
WHERE NOT Emp_City = 'Delhi';
```

4. BETWEEN Operator Selects values **within a range** (inclusive of both endpoints).

```
sql

-- Employees with salary between 30000 and 45000
SELECT * FROM Employee_Details
```

```
WHERE Emp_Salary BETWEEN 30000 AND 45000;
```

5. IN Operator Checks if a value is **in a given list** of values (replaces multiple OR conditions).

```
sql

-- Employees with ID 202, 204, or 205
SELECT * FROM Employee_Details
WHERE Emp_Id IN (202, 204, 205);

-- Opposite: Employees NOT with those IDs
SELECT * FROM Employee_Details
WHERE Emp_Id NOT IN (202, 205);
```

6. LIKE Operator Searches for a **pattern** using wildcards:

- % means any number of characters (zero or more)
- _ means exactly one character

```
sql

-- Names starting with 'S'
SELECT * FROM Employee_Details WHERE Emp_Name LIKE 'S%';

-- Names ending with 'a'
SELECT * FROM Employee_Details WHERE Emp_Name LIKE '%a';

-- Names starting with 'S' and ending with 'a'
SELECT * FROM Employee_Details WHERE Emp_Name LIKE 'S#a';

-- Names where second letter is 'a'
SELECT * FROM Employee_Details WHERE Emp_Name LIKE '_a%';
```

7. ALL Operator Compares a value against **all values** returned by a subquery. Condition must be true for all.

```
sql
```

```
-- Employees earning more than ALL employees in Jaipur
SELECT Emp_Id, Emp_Name FROM Employee_Details
WHERE Emp_Salary > ALL (
    SELECT Emp_Salary FROM Employee_Details
    WHERE Emp_City = 'Jaipur'
);
```

Q33. Create Triggers for ACCOUNT and TRANSACTION_LOG.

Given tables:

- ACCOUNT (CUST_ID, NAME, BALANCE, TYPE, LAST_UPDATED, BRANCH)
- TRANSACTION_LOG (LOG_ID, CUST_ID, OLD_BAL, NEW_BAL, DATE_TIME)

i. Trigger to PREVENT withdrawal if balance becomes negative:

```
sql

CREATE TRIGGER prevent_negative_balance
BEFORE UPDATE ON ACCOUNT
FOR EACH ROW
BEGIN
    IF NEW.BALANCE < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Insufficient balance! Withdrawal denied.';
    END IF;
END;
```

Explanation:

- Fires BEFORE any UPDATE on ACCOUNT table
- NEW.BALANCE is the balance that would result after the update
- If this new balance would be negative, the trigger raises an error
- The UPDATE is blocked — no withdrawal happens
- The bank is protected from overdraft situations

How it works in practice:

Customer has BALANCE = 500
Customer tries to withdraw 700
Update: BALANCE = 500 - 700 = -200
Trigger checks: NEW.BALANCE = -200 < 0 → ERROR!
Withdrawal is rejected.

ii. Trigger to RECORD transaction details in TRANSACTION_LOG:

```
sql

CREATE TRIGGER log_transaction
AFTER UPDATE ON ACCOUNT
FOR EACH ROW
BEGIN
    IF OLD.BALANCE <> NEW.BALANCE THEN
        INSERT INTO TRANSACTION_LOG
            (CUST_ID, OLD_BAL, NEW_BAL, DATE_TIME)
        VALUES
            (OLD.CUST_ID, OLD.BALANCE, NEW.BALANCE, NOW());
    END IF;
END;
```

Explanation:

- Fires AFTER a successful UPDATE on ACCOUNT
- Only logs if the balance actually changed (IF OLD.BALANCE <> NEW.BALANCE)
- OLD.BALANCE = balance before transaction
- NEW.BALANCE = balance after transaction
- NOW() records the exact date and time
- Creates a complete audit trail of all transactions

Result in TRANSACTION_LOG after a ₹200 withdrawal:

LOG_ID	CUST_ID	OLD_BAL	NEW_BAL	DATE_TIME
1	101	1000	800	2025-01-15 14:30:00

Q34. Explain Different Normal Forms in DBMS.

Normalization is the process of organizing a database to reduce data redundancy and avoid update anomalies. Normal Forms are the standards/levels of this organization.

1NF — First Normal Form

Rule: Every column must contain **atomic (indivisible) values** — no repeating groups or multivalued attributes in a single cell.

Violation Example:

SID	NAME	PHONE_NUMBERS
1	Raj	9876543210, 9123456789

PHONE_NUMBERS has multiple values in one cell — this violates 1NF.

Fixed (1NF):

SID	NAME	PHONE
1	Raj	9876543210
1	Raj	9123456789

A relation is in 1NF if:

- All attributes have atomic values
 - No repeating groups
 - Each column has a unique name
 - No duplicate rows (has a primary key)
-

2NF — Second Normal Form

Rule: Must be in 1NF AND every **non-prime attribute must be fully functionally dependent on the entire primary key** (no partial dependencies).

This only matters when the primary key is a **composite key** (made of 2+ attributes).

Violation Example:

EMP_PROJ(EMP_ID, PROJ_ID, HOURS, ENAME, PNAME)

- Primary Key: {EMP_ID, PROJ_ID}
- EMP_ID → ENAME (partial dependency — ENAME depends only on part of the key)
- PROJ_ID → PNAME (partial dependency)
- {EMP_ID, PROJ_ID} → HOURS (full dependency ✓)

Fixed (2NF) — Remove partial dependencies:

- EP1(EMP_ID, PROJ_ID, HOURS) — Full dependency ✓
- EP2(EMP_ID, ENAME) — Remove partial dep for ENAME
- EP3(PROJ_ID, PNAME) — Remove partial dep for PNAME

A relation is in 2NF if:

- It is in 1NF
- No non-prime attribute is partially dependent on any candidate key

3NF — Third Normal Form

Rule: Must be in 2NF AND there must be **no transitive dependencies** (non-prime attribute determining another non-prime attribute).

Violation Example:

EMP_DEPT(EMP_ID, ENAME, DEPT_ID, DEPT_NAME)

- EMP_ID → DEPT_ID (direct)
- DEPT_ID → DEPT_NAME (DEPT_NAME depends on DEPT_ID, not EMP_ID)
- Therefore EMP_ID → DEPT_NAME transitively — violation!

Fixed (3NF):

- ED1(EMP_ID, ENAME, DEPT_ID) — Direct dependencies ✓
- ED2(DEPT_ID, DEPT_NAME) — Separate out the transitive dependency

A relation is in 3NF if:

- It is in 2NF
- For every non-trivial FD $X \rightarrow A$: either X is a superkey OR A is a prime attribute

Simple test: No non-prime attribute should determine another non-prime attribute.

BCNF — Boyce-Codd Normal Form

Rule: Must be in 3NF AND for every non-trivial FD $X \rightarrow Y$, **X must be a superkey**.

BCNF is stricter than 3NF. It removes cases where prime attributes have functional dependencies from non-superkey attributes.

Example where 3NF holds but BCNF doesn't:

TEACH(STUDENT, COURSE, INSTRUCTOR)

- FD1: {STUDENT, COURSE} $\rightarrow$ INSTRUCTOR
- FD2: INSTRUCTOR $\rightarrow$ COURSE (instructor teaches one course)
- Candidate Keys: {STUDENT, COURSE} and {STUDENT, INSTRUCTOR}
- INSTRUCTOR $\rightarrow$ COURSE: INSTRUCTOR is not a superkey but COURSE is prime $\leftarrow$ Allowed in 3NF but not BCNF!

A relation is in BCNF if:

- It is in 3NF
- For every non-trivial FD $X \rightarrow Y$: X must be a superkey

BCNF is stronger than 3NF: Every BCNF relation is in 3NF, but not every 3NF relation is in BCNF.

Summary of Normal Forms:

Normal Form	Condition to Satisfy
1NF	All attributes atomic, no repeating groups
2NF	1NF + No partial dependencies on composite key
3NF	2NF + No transitive dependencies
BCNF	3NF + Every determinant is a superkey

General rule: Higher normal form = less redundancy = less risk of anomalies.

Q35. Find Closure, Candidate Keys, and Superkeys for R(A,B,C,D,E) with FDs: $AB \rightarrow C$, $AC \rightarrow D$, $D \rightarrow E$.

Given FDs:

- $AB \rightarrow C$
 - $AC \rightarrow D$
 - $D \rightarrow E$
-

i) Find Closure of AB^+ and AC^+ :

AB^+ :

- Start: $\{A, B\}$
- $AB \rightarrow C \rightarrow$ add $C \rightarrow \{A, B, C\}$
- $AC \rightarrow D$ (have A and C) $\rightarrow$ add $D \rightarrow \{A, B, C, D\}$
- $D \rightarrow E \rightarrow$ add $E \rightarrow \{A, B, C, D, E\}$
- $AB^+ = \{A, B, C, D, E\}$ ✓ Covers all attributes

AC^+ :

- Start: $\{A, C\}$
 - $AC \rightarrow D \rightarrow$ add $D \rightarrow \{A, C, D\}$
 - $D \rightarrow E \rightarrow$ add $E \rightarrow \{A, C, D, E\}$
 - $AB \rightarrow C?$ (need B, don't have it) $\rightarrow$ can't apply
 - $AC^+ = \{A, C, D, E\}$ — Does NOT cover B $\rightarrow$ Not a superkey
-

ii) Find All Candidate Keys:

Check AB: $AB^+ = \{A, B, C, D, E\}$ ✓ Superkey

- A alone? A^+ : A only, nothing fires $\rightarrow \{A\}$ — Not superkey
- B alone? B^+ : B only, nothing fires $\rightarrow \{B\}$ — Not superkey
- AB has no proper subset that is a superkey $\rightarrow$ **AB is a Candidate Key** ✓

Check other possibilities — we need to include B since B doesn't appear on any RHS:

- B never appears on the right side of any FD
- So B must be in every candidate key (otherwise B can't be determined)
- Similarly, A never appears on the right side — A must also be in every candidate key

Since both A and B must be in every key, and AB alone forms a superkey, **AB is the only candidate key.**

Candidate Key: AB

iii) Find Some Superkeys:

All supersets of AB are superkeys:

- AB
- ABC
- ABD
- ABE
- ABCD
- ABCE
- ABDE
- ABCDE

All 8 combinations that include both A and B are superkeys.

Q36. Explain the ACID Properties of Transactions with Suitable Examples.

(This is same as Q12 — ACID properties — see Q12 answer above for complete detailed answer)

Quick Summary:

- **Atomicity** — All or nothing (bank transfer: both debit and credit happen, or neither)
 - **Consistency** — Valid state to valid state (balance can't go below minimum)
 - **Isolation** — Transactions don't interfere (two users booking same seat — only one succeeds)
 - **Durability** — Committed changes are permanent (even if server crashes after commit)
-

Q37. Illustrate the Two-Phase Locking (2PL) Protocol.

Two-Phase Locking (2PL) is a concurrency control protocol that ensures conflict serializability. It controls when transactions can acquire and release locks on data items.

What is a Lock?

A lock is a mechanism that controls access to a data item:

- **Shared Lock (S-lock / Read Lock)** — Multiple transactions can read simultaneously. Used for READ operations.
- **Exclusive Lock (X-lock / Write Lock)** — Only one transaction can hold this. Blocks all others. Used for WRITE operations.

Lock Compatibility:

	S-Lock held	X-Lock held
Request S-Lock	✓ Compatible	✗ Wait
Request X-Lock	✗ Wait	✗ Wait

The Two Phases of 2PL:

Phase 1: Growing Phase (Lock Acquisition Phase)

- Transaction can **acquire (lock) new locks** on data items
- Transaction **cannot release any locks** during this phase
- The transaction keeps collecting more and more locks

Phase 2: Shrinking Phase (Lock Release Phase)

- Transaction can **release (unlock) locks**
- Transaction **cannot acquire any new locks** during this phase
- Once you release the first lock, you can never get a new lock

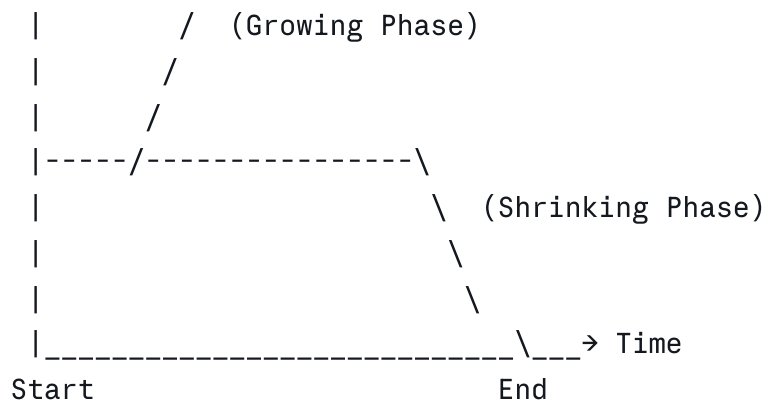
Lock Point: The moment when the transaction has acquired ALL its locks and is about to start releasing them. This is the boundary between the two phases.

Diagram of 2PL:

Number of
Locks held

| Lock Point

| /



Example of 2PL:

Transaction T1 wants to transfer money from Account A to Account B:

Phase 1 (Growing):

T1 acquires X-lock on Account A
 T1 reads Account A balance
 T1 updates Account A (deduct ₹500)
 T1 acquires X-lock on Account B
 ← Lock Point reached ←

Phase 2 (Shrinking):

T1 reads Account B balance
 T1 updates Account B (add ₹500)
 T1 releases X-lock on Account A
 T1 releases X-lock on Account B
 T1 COMMIT

Types of 2PL:

1. **Basic 2PL** Standard 2PL as described above. Prevents some but not all problems.
2. **Strict 2PL** Holds ALL exclusive (write) locks until the transaction commits or aborts. Prevents dirty reads.
 - Most commonly used in practice
3. **Rigorous 2PL** Holds ALL locks (both shared and exclusive) until commit or abort.
 - Strictest form, ensures full isolation

4. Conservative 2PL (Static 2PL) Transaction acquires ALL locks it will need before it starts executing.

- Prevents deadlocks but requires knowing all needed locks in advance (impractical)
-

Advantages of 2PL:

- Guarantees conflict serializability
- Prevents lost updates and dirty reads

Disadvantages of 2PL:

- Can cause **deadlocks** (two transactions waiting for each other's locks)
 - Reduces concurrency (other transactions must wait)
 - **Cascading rollbacks** can occur in basic 2PL
-

Q36(b). Describe Different Types of OLAP Operations.

(This is a repeat — same as Part-A Q29 but in more detail for 10 marks)

OLAP (Online Analytical Processing) is a technology that allows users to analyze data from multiple dimensions. It operates on a **Data Cube** — a multi-dimensional representation of data.

The Data Cube Concept:

A data cube has multiple dimensions. For example, a Sales data cube might have:

- **Dimension 1:** Time (Day, Month, Quarter, Year)
 - **Dimension 2:** Location (City, State, Country)
 - **Dimension 3:** Product (Item, Category, Brand)
 - **Measure:** Sales Amount (the value being analyzed)
-

OLAP Operations:

1. Roll-Up (Aggregation / Drill-Up)

Moves **from detailed data to more summarized data** by climbing up the dimension hierarchy.

Example:

- Start with: Sales per City (Delhi=₹50L, Mumbai=₹80L, Chennai=₹30L)
- Roll-Up to: Sales per State (Maharashtra=₹80L, Tamil Nadu=₹30L, Delhi=₹50L)
- Roll-Up further: Sales per Country (India=₹160L)

```
sql
```

```
SELECT YEAR, SUM(SALES) AS Total_Sales
FROM Sales_Data
GROUP BY YEAR;
-- Instead of month-wise data, we see year-wise (rolled up)
```

Use: Management wants summary — "What are total annual sales?"

2. Drill-Down (Disaggregation)

The **opposite of Roll-Up**. Moves from summary to more detailed data.

Example:

- Start with: Annual sales = ₹160L
- Drill-Down to: Quarterly sales (Q1=₹30L, Q2=₹45L, Q3=₹50L, Q4=₹35L)
- Drill-Down further: Monthly sales for each quarter

Use: Analyst wants details — "Which specific month caused the sales spike in Q3?"

3. Slice

Selects **one specific value for one dimension** and creates a 2D subset (like slicing a bread loaf).

Example: From a 3D cube (Time × Location × Product):

- Slice on Time = "Year 2023"
- Result: A 2D table of Location × Product sales for only 2023

```
sql
```

```
SELECT LOCATION, PRODUCT, SUM(SALES)
FROM Sales_Data
```

```
WHERE YEAR = 2023
GROUP BY LOCATION, PRODUCT;
```

Use: "Show me all sales data but only for year 2023."

4. Dice

Selects **specific values for two or more dimensions** — creates a smaller sub-cube.

Example: From the same cube:

- Dice on Time = [2022, 2023] AND Location = [North India, South India]
- Result: A sub-cube with only those combinations

```
sql

SELECT YEAR, REGION, PRODUCT, SUM(SALES)
FROM Sales_Data
WHERE YEAR IN (2022, 2023)
AND REGION IN ('North India', 'South India')
GROUP BY YEAR, REGION, PRODUCT;
```

Difference from Slice: Slice = one value of one dimension. Dice = multiple values across multiple dimensions.

5. Pivot (Rotate)

Rotates the data cube to view it from a different angle. Swaps rows and columns to give a different perspective.

Example:

- Original view: Rows = Products, Columns = Years
- After Pivot: Rows = Years, Columns = Products

	2021	2022	2023
Laptop	100	150	200
Phone	200	250	300

After Pivot:

	Laptop	Phone
2021	100	200
2022	150	250
2023	200	300

Use: Change perspective without changing the underlying data — useful for cross-tab reports.

Summary Table:

Operation	What it Does	Analogy
Roll-Up	Detailed → Summary	Zoom out on a map
Drill-Down	Summary → Detailed	Zoom in on a map
Slice	Pick one value of one dimension	Cut a slice of bread
Dice	Pick ranges of multiple dimensions	Cut a small cube from a larger cube
Pivot	Rotate/transpose the view	Rotate a cube to see another face

Q37(b). Illustrate Testing for Conflict Serializability with an Algorithm.

(Repeat of Q11 — see Q11 answer above for complete detailed answer with precedence graph example)

Quick Summary:

1. Create a node for each transaction
2. For each conflicting pair (Read-Write, Write-Read, Write-Write on same data item):
 - If T_i 's operation comes BEFORE T_j 's conflicting operation → draw arrow $T_i \rightarrow T_j$
3. Check the graph:
 - **No cycle** → Conflict Serializable ✅
 - **Cycle exists** → NOT Conflict Serializable ❌

Q38. Explain Desirable Properties of Transactions in DBMS.

(Repeat of Q12 — ACID Properties — see Q12 answer above for complete detailed answer)

Quick Summary with examples:

Atomicity: Bank transfer of ₹5000 — either both debit and credit happen, or neither happens. No half-done transactions.

Consistency: All database rules and constraints must hold before and after every transaction. Balance cannot go below ₹0 (if that's a rule).

Isolation: Multiple users booking the last available seat — only one succeeds. Each transaction behaves as if it runs alone.

Durability: After a transfer is confirmed (committed), even a server crash cannot undo it. Changes are permanent.

All 38 Part-B questions (Q14 to Q38) answered completely! This completes the entire question bank — all 29 Part-A and all 38 Part-B questions.

